
SKILLEVOLBENCH: Benchmarking the Evolution from Episodic Experience to Procedural Skills

Anonymous Author(s)

Affiliation

Address

email

Abstract

Large language model (LLM) agents accumulate rich episodic trajectories while solving real-world tasks, but it remains unclear whether such experience can be distilled into reusable procedural skills. We introduce **SKILLEVOLBENCH**, a diagnostic benchmark for evaluating this step from experience reuse to skill formation. It contains 180 tasks across six real-world agent environments, organized into role-conditioned task families with shared latent procedures. Agents learn from acquisition tasks, update an external skill library using compacted trajectories and verifier feedback, and then face frozen deployment tasks testing transfer under context shift, adversarial shortcuts, and composition. By comparing self-generated and curated-start skill evolution against no-skill and raw-trajectory controls, SKILLEVOLBENCH separates procedural abstraction from base capability, curated prior knowledge, and direct reuse of episodic traces. Across ten model configurations and three agent harnesses, we find that current agents often adapt locally but rarely form robust reusable skills. Skill-based conditions can improve acquisition or replay, and individual models sometimes gain on specific deployment axes. However, these gains are unstable under context shift, adversarial shortcuts, and composition. Raw-trajectory reuse frequently outperforms distilled skills, suggesting that current abstraction procedures discard contextual and procedural cues that remain useful for future tasks. More frequent skill updating is also not monotonically beneficial, as additional revisions can improve coverage while introducing episode-specific drift. These findings highlight SKILLEVOLBENCH as a testbed for measuring when one-off experience becomes durable procedural knowledge.

1 Introduction

Large language model (LLM) agents are increasingly being deployed as practical interfaces for real-world tasks. Unlike static question-answering systems, these agents interact with external environments over multi-step trajectories by reasoning, calling tools, inspecting files, executing code, and observing feedback [1, 2, 3, 4]. As agents act over longer horizons, each task attempt leaves behind an episodic trajectory that records how the attempt unfolded. Prior work has shown that such experience can be stored and reused in later tasks [5, 6, 7, 8]. Yet reusing an episode is not the same as extracting a procedure. A trajectory records what happened once and often mixes transferable decisions with incidental details, failed hypotheses, and mistakes. Future tasks rarely repeat the same episode exactly. They require a more explicit procedural form that states what to do again, when to do it, and what to check along the way. This is the role that *agent skills* [9, 10] are beginning to play: they turn reusable know-how into an external artifact that can be loaded when relevant and followed by future agents.

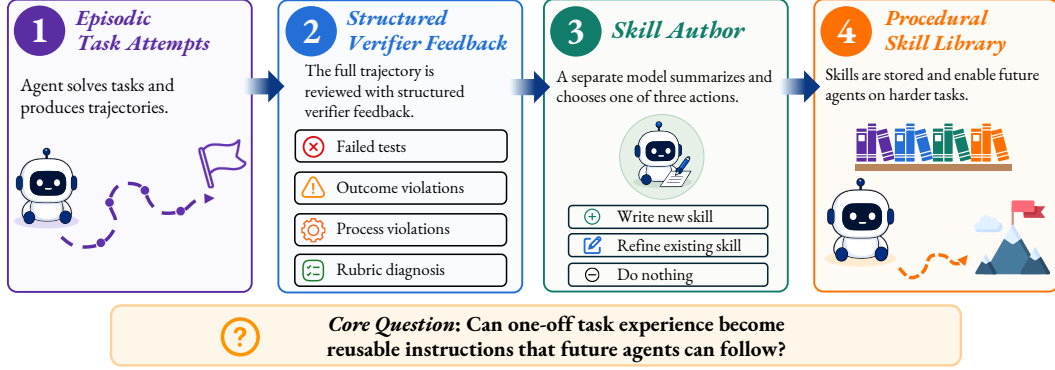


Figure 1: **SKILLEVOLBENCH probes whether episodic agent experience can be abstracted into reusable procedural skills.** During acquisition, each eligible attempt produces logged execution artifacts that are compacted into a structured trajectory summary and paired with verifier feedback. A host-side Skill Author call, separated from the task-solving agent loop, uses this evidence and the current family skill state to write, refine, or skip a library update. The resulting library is frozen for harder deployment tasks, isolating procedural reuse from raw trajectory replay and test-time repair.

SkillsBench [11] recently shows that curated skills improve agent performance across diverse domains, while self-generated skills provide little benefit on average. However, its self-generated setting is cold-start: agents write procedural guidance before attempting the task or observing verifier feedback. This leaves open the central gap between *skill use* and *skill formation*. If curated skills show that procedural knowledge is useful, and experience-reuse methods show that trajectories contain task-solving evidence, can agents distill noisy one-off experience into compact skills that future agents can load, follow, and apply beyond the original episode?

To study this question, we introduce **SKILLEVOLBENCH**, a diagnostic benchmark for the missing step between episodic experience and procedural reuse. As shown in Figure 1, the benchmark turns each learning attempt into an abstraction step: an episodic trajectory and structured verifier feedback are passed to a host-side Skill Author call, which decides whether to write a new skill, revise an existing one, or leave the library unchanged. The resulting library is then frozen before harder related tasks are evaluated, so success depends on whether noisy one-off experience has already been encoded as reusable procedure. **SKILLEVOLBENCH** spans real-world work from engineering workflows to information work and workplace operations. It contains six environments, each with five task families, where each family shares a latent procedural pattern across related problems. Within each family, three learning tasks move from a canonical episode to targeted variants that expose the limits of a naive procedure, and three frozen evaluation tasks test transfer under context shift, adversarial shortcuts, and multi-skill composition. Performance therefore reflects whether the agent has extracted what should generalize from noisy episodes before harder related tasks are seen.

We evaluate this question in both **SELF-GENERATED** and **CURATED-START** settings. The **SELF-GENERATED** setting tests whether agents can induce skills from their own learning episodes, while the **CURATED-START** setting tests whether experience can improve human-written procedural priors. We compare both against **NO-SKILL** and **RAW-TRAJECTORY** controls, and replay the original learning tasks with the final frozen library to separate local recovery from deployment transfer.

This design makes three aspects of skill evolution measurable. First, **SKILLEVOLBENCH** evaluates skill formation rather than only skill use: each family requires agents to convert verifier-grounded episodes into a persistent procedural artifact before harder related tasks are seen. Second, its role-conditioned task arcs separate acquisition, replay, transfer, context shift, adversarial robustness, and composition, exposing failure modes that a single success rate would hide. Third, its controls distinguish procedural abstraction from base agent capability, curated prior knowledge, and direct reuse of raw episodic traces.

Across ten model configurations and three agent harnesses, we find that current agents exhibit local procedural adaptation but not reliable reusable skill formation. Skill-based agents can improve acquisition or replay, yet these gains do not consistently transfer to frozen deployment tasks. **RAW-**

TRAJECTORY controls further reveal a lossy abstraction bottleneck: agents often use episodic traces more effectively than the distilled skills derived from them. Together, these results position SKILLEVOLBENCH as a diagnostic testbed for studying when experience becomes a reusable skill, when it remains an episode-specific patch, and when skill abstraction itself loses information needed for future tasks.

2 Related Work

From static tasks to realistic agent work. Agent benchmarks have increasingly moved from static tasks toward interactive settings that resemble real-world work [12, 13]. Mind2Web, MindWeb, and WebArena evaluate multi-step web navigation [3, 14, 15]; SWE-bench grounds software-engineering evaluation in real GitHub issues [4]; and OSWorld, τ -bench, and TheAgentCompany extend evaluation to computer use, user interaction, tool policies, and workplace workflows [16, 17, 18]. These benchmarks make agent evaluation more realistic, but they mainly measure whether an agent can complete a task rather than whether experience becomes reusable skill.

Reusing agent experience. A growing line of work studies how agents improve by reusing prior experience without updating model parameters. Reflexion stores verbal feedback in episodic memory [5], ExpeL extracts lessons from accumulated experiences [6], Synapse retrieves complete past trajectories as exemplars [7], and Agent Workflow Memory induces reusable workflows from web-agent executions [8]. These methods show that trajectories and reflections contain useful task-solving evidence, but they primarily reuse episodic traces or derived lessons rather than evaluating whether such evidence becomes durable procedural artifacts.

Agent Skills and skill evolution. Agent Skills make procedural knowledge explicit by packaging task guidance, scripts, references, and resources into loadable artifacts [9, 10]. SkillsBench shows that curated skills can improve performance, while cold-start self-generated skills provide limited average gains [11]. Related systems study LLM-generated tools [19, 20], executable code-skill libraries [21], and skill discovery, memory skills, self-evolution, or trajectory-derived skill libraries [22, 23, 24, 25, 26, 27, 28, 29, 30]. SKILLEVOLBENCH complements this work by evaluating whether task-local episodes and verifier feedback can produce external skill artifacts that survive frozen transfer.

3 SKILLEVOLBENCH

3.1 Overview

SKILLEVOLBENCH evaluates whether agents can transform repeated task experience into reusable procedural skills. It contains 180 tasks across six real-world agent environments, with five task families per environment and six role-conditioned tasks per family. Each family defines a skill-evolution arc: tasks share an underlying procedure but vary failure modes, surface forms, and deployment conditions. This design distinguishes task-specific fixes from skills that can be revised, invoked, and composed.

3.2 Environments and Task Families

Fig. 2 summarizes the taxonomy. The six environments cover common forms of agent work: code modification, API orchestration, data processing, document transformation, research synthesis, and communication operations. A task family corresponds to a recurring procedural capability rather than a topic label. Families are therefore related enough for experience to matter, but different enough to separate procedural learning from memorization.

3.3 Task Construction

Task selection. We construct SKILLEVOLBENCH through a source-driven and human-curated process. We do not reuse existing benchmark instances. Instead, we use open-source agent skill collections, skill-oriented benchmarks, and practitioner-facing examples as evidence for task topics and workflow motifs [10, 11, 31, 32, 33]. These sources guide the design space but do not define the tasks directly. We cluster observed workflows by artifact type, required tools, interaction pattern, and

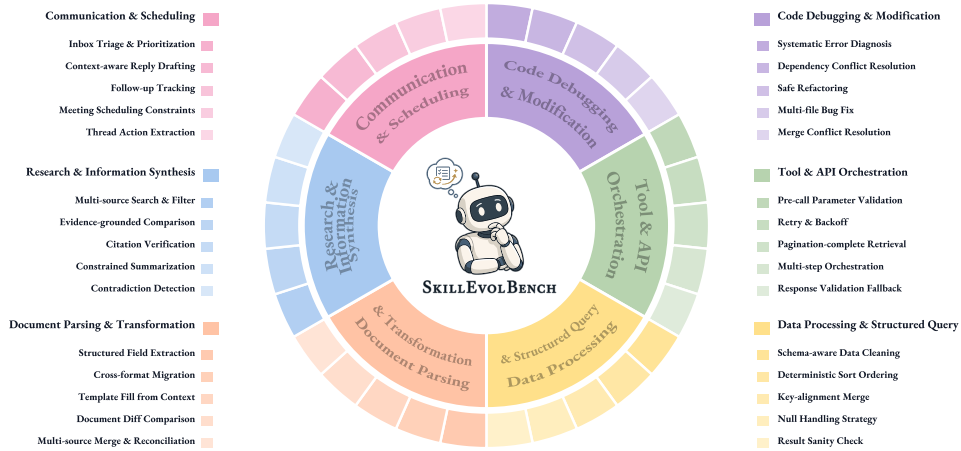


Figure 2: **SKILLEVOLBENCH organizes agent work into controlled skill-evolution arcs.** The benchmark spans 180 tasks across six real-world environments, with five recurring procedural families in each environment. Each family follows the same six-role progression: canonical, enriched, and variant tasks expose and stress-test the target procedure, while context-shift, adversarial, and composition tasks evaluate whether the evolved skill transfers, resists shortcuts, and combines with other skills in realistic workflows.

119 solution procedure, then retain families that satisfy three desiderata: real-world relevance, procedural
 120 skill fit, and verifiable evolvability. In particular, each family must describe a reusable procedure
 121 that is specific enough to be written as a skill, general enough to transfer beyond one fixture, and
 122 evaluable through deterministic outcome checks and process-level evidence.

123 **Role-conditioned progression.** For each family, we instantiate six roles. The first three support
 124 skill acquisition: the *canonical* task presents the base procedure, the *enriched* task exposes a missing
 125 sub-capability, and the *variant* task changes the surface form while preserving the same procedure.
 126 The last three evaluate deployment: the *context-shift* task embeds the skill need in a broader request,
 127 the *adversarial* task introduces shortcut solutions that can pass shallow checks, and the *composition*
 128 task requires the target skill to interact with other skills. This progression tests transfer, implicit
 129 invocation, shortcut resistance, and composition within each family.

130 **Gap-exposed curated skills.** For each family, we provide a *gap-exposed curated skill*. It is neither
 131 an oracle solution nor copied from any task instance. We first define the family-level procedure and the
 132 gaps that should remain exposed. A *skill-creator* drafts an initial skill from this specification [34],
 133 and we manually refine the draft to control its granularity. The resulting skill should support the
 134 canonical task while leaving enriched, variant, adversarial, and compositional cases unsolved, giving
 135 curated-start agents a useful but bounded initialization.

136 **Specification and review.** Each task contains an instruction and fixture, a verification suite, and
 137 a scoring rubric. The verification suite includes public tests for the basic contract, hidden tests for
 138 edge cases and distribution shifts, and process verifiers that inspect traces and artifacts for brittle
 139 strategies such as hard-coded constants, swallowed exceptions, skipped validation, or incomplete
 140 repairs. Before inclusion, each family and curated skill is manually reviewed for realism, role
 141 alignment, verifier coverage, and whether the curated skill is useful but incomplete. The complete
 142 specifications are provided in Appendix Secs. A and B.

143 4 Skill Evolution Protocol

144 4.1 Protocol Overview

145 Each environment in SKILLEVOLBENCH is evaluated as an independent lifelong episode. As shown
 146 in Fig. 3, an episode activates a fresh environment-scoped skill library. The agent first completes
 147 acquisition tasks, where logged execution artifacts are compacted and paired with verifier feedback as
 148 evidence for possible skill updates; implementation details of trajectory compaction are provided in
 149 Appendix Sec. C.2. The resulting library is then frozen for deployment. When enabled, replay reruns
 150 the original acquisition tasks with the final frozen library. Moving to a new environment activates a
 151 fresh library; skills from previous environments are retained only for logging and audit and are not
 152 mounted for the next episode.

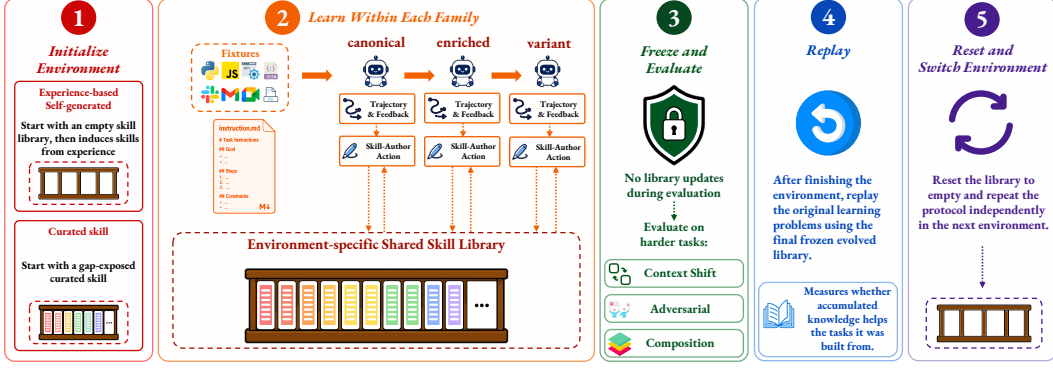


Figure 3: **SKILLEVOLBENCH evaluation protocol.** Each environment forms a self-contained lifelong episode. Agents learn from canonical, enriched, and variant tasks, using trajectories and verifier feedback to update an environment-specific skill library. The library is then frozen for context-shift, adversarial, and composition evaluation. Replay measures whether the evolved library also improves the original learning tasks, after which the library is reset before the next environment.

4.2 Initialization and Skill Conditions

We compare three ways of initializing family-level procedural knowledge. In the *experience-based self-generated* condition, a family starts with no skill; the *canonical* task is attempted without a family skill, and induction may occur only after execution evidence and verifier feedback are available. In the *zero-shot generated* condition, a metadata-only skill is generated before execution and remains fixed. In the *curated* condition, a family starts from a *gap-exposed curated skill*, which covers the base procedure but leaves room for acquisition tasks to expose missing sub-capabilities. The curated seed is fixed in static variants and may be refined only when revision is enabled.

For environment e and family f , the starting skill set under condition c is

$$S_c(e, f) = \begin{cases} \emptyset, & c = \text{exp-self}, \\ \{\hat{s}_{e,f}^0\}, & c = \text{zero-shot}, \\ \{s_{e,f}^{\text{gap}}\}, & c = \text{curated}. \end{cases} \quad (1)$$

Here $\hat{s}_{e,f}^0$ is the zero-shot skill and $s_{e,f}^{\text{gap}}$ is the curated seed from sec. 3.3. Full authoring prompts are provided in sec. C.3.

4.3 Acquisition: From Episodic Evidence to Skill Updates

Let $x_{e,f}^r$ denote the task in environment e , family f , and role r . Acquisition uses the *canonical*, *enriched*, and *variant* roles:

$$\mathcal{R}_{\text{acq}} = \{\text{can}, \text{enr}, \text{var}\}. \quad (2)$$

All acquisition tasks in an environment are completed before deployment begins. The active library is scoped to the environment, so skills learned from earlier families may be visible to later families in the same environment, but never transfer across environments. A family-level starting skill $S_c(e, f)$, when present, is introduced when that family's *canonical* task is first reached.

Each acquisition attempt yields a compacted trajectory summary $\tilde{\tau}_{e,f}^r$ from harness-recorded artifacts such as instructions, file accesses, tool calls, commands, edits, generated outputs, tests, and final responses. The verifier returns feedback $v_{e,f}^r$, including outcome results, process checks, rewards, and diagnostics. We do not access hidden model state or private chain-of-thought.

Skill authoring is family-local. Although the task-solving agent may read the environment-level library, the Skill Author receives only same-family skills and same-family acquisition history. With $\text{can} \prec \text{enr} \prec \text{var}$, the available evidence after role r is

$$H_{e,f}^{\leq r} = ((\tilde{\tau}_{e,f}^{r'}, v_{e,f}^{r'}))_{r' \in \mathcal{R}_{\text{acq}}, r' \preceq r}. \quad (3)$$

Table 1: **NO-SKILL vs. CURATED-START skills.** Values are success rates (%). Deltas report percentage-point differences from the corresponding NO-SKILL result. For RSR, the reference is NO-SKILL LSR because NO-SKILL has no replay phase. Red/blue/gray denote positive/negative/zero deltas.

Agent Harness	Model	Condition	LSR		RSR		ESR		CSSR		ARSR		CompSR	
			%	Δ	%	Δ	%	Δ	%	Δ	%	Δ	%	Δ
CLAUDE CODE	Claude Opus 4.6	NO-SKILL	38.9	-	-	-	37.8	-	46.7	-	36.7	-	30.0	-
		CURATED-STATIC	40.0	+1.1	-	-	34.4	-3.4	46.7	0.0	36.7	0.0	20.0	-10.0
		+ REVISION	44.4	+5.5	56.7	+17.8	38.9	+1.1	40.0	-6.7	43.3	+6.6	33.3	+3.3
	Claude Opus 4.5	+ ALWAYS	42.2	+3.3	46.7	+7.8	37.8	0.0	40.0	-6.7	46.7	+10.0	26.7	-3.3
		NO-SKILL	42.2	-	-	-	32.2	-	40.0	-	40.0	-	16.7	-
		CURATED-STATIC	45.6	+3.4	-	-	34.4	+2.2	43.3	+3.3	40.0	0.0	20.0	+3.3
	Claude Sonnet 4.6	+ REVISION	38.9	-3.3	42.2	0.0	34.4	+2.2	43.3	+3.3	36.7	-6.7	23.3	+6.6
		+ ALWAYS	42.2	0.0	44.4	+2.2	36.7	+4.5	43.3	+3.3	43.3	+3.3	23.3	+6.6
	Claude Sonnet 4.5	NO-SKILL	37.8	-	-	-	38.9	-	40.0	-	50.0	-	26.7	-
		CURATED-STATIC	41.1	+3.3	-	-	35.6	-3.3	36.7	-3.3	46.7	-3.3	23.3	-3.4
		+ REVISION	41.1	+3.3	51.1	+13.3	38.9	0.0	46.7	+6.7	43.3	-6.7	26.7	0.0
		+ ALWAYS	40.0	+2.2	44.4	+6.6	38.9	0.0	46.7	+6.7	43.3	-6.7	26.7	0.0
CODEX CLI	GPT-5.4	NO-SKILL	41.1	-	-	-	35.6	-	40.0	-	46.7	-	20.0	-
		CURATED-STATIC	35.6	-5.5	-	-	36.7	+1.1	43.3	+3.3	43.3	-3.4	23.3	+3.3
		+ REVISION	40.0	-1.1	42.2	+1.1	28.9	-6.7	36.7	-3.3	33.3	-13.4	16.7	-3.3
	GPT-5.3-Codex	+ ALWAYS	37.8	-3.3	38.9	-2.2	33.3	-2.3	40.0	0.0	40.0	-6.7	20.0	0.0
		NO-SKILL	43.3	-	-	-	33.3	-	43.3	-	33.3	-	23.3	-
		CURATED-STATIC	43.3	0.0	-	-	32.2	-1.1	40.0	-3.3	40.0	+6.7	16.7	-6.6
	GPT-5.2-Codex	+ REVISION	45.6	+2.3	45.6	+2.3	38.9	+5.6	50.0	+6.7	43.3	+10.0	23.3	0.0
		+ ALWAYS	43.3	0.0	42.2	-1.1	40.0	+6.7	40.0	-3.3	50.0	+16.7	30.0	+6.7
	GPT-5.2-Codex	NO-SKILL	44.4	-	-	-	34.4	-	36.7	-	46.7	-	20.0	-
		CURATED-STATIC	45.6	+1.2	-	-	32.2	-2.2	36.7	0.0	43.3	-3.4	16.7	-3.3
		+ REVISION	42.2	-2.2	48.9	+4.5	34.4	0.0	36.7	0.0	46.7	0.0	20.0	0.0
		+ ALWAYS	42.2	-2.2	45.6	+1.2	32.2	-2.2	40.0	+3.3	40.0	-6.7	16.7	-3.3
GEMINI CLI	Gemini 3.1 Pro	NO-SKILL	43.3	-	-	-	36.7	-	40.0	-	53.3	-	16.7	-
		CURATED-STATIC	51.1	+7.8	-	-	30.0	-6.7	40.0	0.0	36.7	-16.6	13.3	-3.4
		+ REVISION	45.6	+2.3	43.3	0.0	36.7	0.0	43.3	+3.3	46.7	-6.6	20.0	+3.3
	Gemini 3 Flash	+ ALWAYS	44.4	+1.1	45.6	+2.3	37.8	+1.1	46.7	+6.7	43.3	-10.0	23.3	+6.6
		NO-SKILL	40.0	-	-	-	35.6	-	40.0	-	46.7	-	20.0	-
		CURATED-STATIC	44.4	+4.4	-	-	35.6	0.0	36.7	-3.3	43.3	-3.4	26.7	+6.7
	Gemini 2.5 Pro	+ REVISION	43.3	+3.3	55.6	+15.6	35.6	0.0	36.7	-3.3	43.3	-3.4	26.7	+6.7
		+ ALWAYS	40.0	0.0	45.6	+5.6	35.6	0.0	40.0	0.0	40.0	-6.7	26.7	+6.7
		NO-SKILL	40.0	-	-	-	35.6	-	30.0	-	53.3	-	23.3	-
	Gemini 3 Flash	CURATED-STATIC	41.1	+1.1	-	-	32.2	-3.4	36.7	+6.7	43.3	-10.0	16.7	-6.6
		+ REVISION	42.2	+2.2	42.2	+2.2	32.2	-3.4	30.0	0.0	46.7	-6.6	20.0	-3.3
		+ ALWAYS	43.3	+3.3	37.8	-2.2	37.8	+2.2	46.7	+16.7	36.7	-16.6	30.0	+6.7
	Gemini 2.5 Pro	NO-SKILL	30.0	-	-	-	26.7	-	26.7	-	30.0	-	23.3	-
		CURATED-STATIC	30.0	0.0	-	-	18.9	-7.8	20.0	-6.7	23.3	-6.7	13.3	-10.0
		+ REVISION	28.9	-1.1	32.2	+2.2	21.1	-5.6	16.7	-10.0	30.0	0.0	16.7	-6.6
		+ ALWAYS	31.1	+1.1	26.7	-3.3	24.4	-2.3	26.7	0.0	23.3	-6.7	23.3	0.0

178 The host-side Skill Author is invoked only after eligible acquisition attempts and emits a structured
 179 library edit:

$$L_{e,k+1} = U_c \left(L_{e,k}, H_{e,fk}^{<r_k} \right). \quad (4)$$

180

181 The update rule U_c depends on the condition. Experience-based self-generation may induce a new
 182 skill after the *canonical* attempt and revise it on later failed acquisition attempts. Always-update
 183 variants invoke authoring after every eligible acquisition attempt. Curated revision variants refine the
 184 curated seed under the same trigger policy, while curated static keeps it fixed. Zero-shot skills are
 185 never revised:

$$U_{\text{zero}}(L, H) = L. \quad (5)$$

186 4.4 Frozen Deployment and Replay

187 After acquisition, the environment-specific library is frozen. Deployment uses the *context-shift*,
 188 *adversarial*, and *composition* roles. During deployment, the agent may read and apply accumulated

Table 2: **NO-SKILL vs. SELF-GENERATED skills.** Values are success rates (%) with percentage-point deltas relative to NO-SKILL. RSR deltas use NO-SKILL LSR as the baseline. Zero-shot skills are metadata-induced and unrevised.

Agent Harness	Model	Condition	LSR		RSR		ESR		CSSR		ARSR		CompSR	
			%	Δ	%	Δ	%	Δ	%	Δ	%	Δ	%	Δ
CLAUDE CODE	Claude Opus 4.6	NO-SKILL	38.9	-	-	-	37.8	-	46.7	-	36.7	-	30.0	-
		ZERO-SHOT	41.1	+2.2	-	-	32.2	-5.6	40.0	-6.7	33.3	-3.4	23.3	-6.7
		EXPERIENCE	44.4	+5.5	48.9	+10.0	32.2	-5.6	40.0	-6.7	36.7	0.0	20.0	-10.0
		+ ALWAYS	42.2	+3.3	45.6	+6.7	37.8	0.0	43.3	-3.4	46.7	+10.0	23.3	-6.7
	Claude Opus 4.5	NO-SKILL	42.2	-	-	-	32.2	-	40.0	-	40.0	-	16.7	-
		ZERO-SHOT	41.1	-1.1	-	-	34.4	+2.2	43.3	+3.3	40.0	0.0	20.0	+3.3
		EXPERIENCE	42.2	0.0	40.0	-2.2	31.1	-1.1	33.3	-6.7	40.0	0.0	20.0	+3.3
		+ ALWAYS	41.1	-1.1	40.0	-2.2	35.6	+3.4	40.0	0.0	40.0	0.0	26.7	+10.0
	Claude Sonnet 4.6	NO-SKILL	37.8	-	-	-	38.9	-	40.0	-	50.0	-	26.7	-
		ZERO-SHOT	37.8	0.0	-	-	36.7	-2.2	40.0	0.0	43.3	-6.7	26.7	0.0
		EXPERIENCE	44.4	+6.6	46.7	+8.9	40.0	+1.1	46.7	+6.7	40.0	-10.0	33.3	+6.6
		+ ALWAYS	41.1	+3.3	46.7	+8.9	40.0	+1.1	40.0	0.0	50.0	0.0	30.0	+3.3
	Claude Sonnet 4.5	NO-SKILL	41.1	-	-	-	35.6	-	40.0	-	46.7	-	20.0	-
		ZERO-SHOT	41.1	0.0	-	-	27.8	-7.8	33.3	-6.7	30.0	-16.7	20.0	0.0
		EXPERIENCE	36.7	-4.4	41.1	0.0	35.6	0.0	40.0	0.0	40.0	-6.7	26.7	+6.7
		+ ALWAYS	38.9	-2.2	44.4	+3.3	33.3	-2.3	40.0	0.0	40.0	-6.7	20.0	0.0
CODEX CLI	GPT-5.4	NO-SKILL	43.3	-	-	-	33.3	-	43.3	-	33.3	-	23.3	-
		ZERO-SHOT	45.6	+2.3	-	-	33.3	0.0	43.3	0.0	40.0	+6.7	16.7	-6.6
		EXPERIENCE	45.6	+2.3	44.4	+1.1	31.1	-2.2	40.0	-3.3	36.7	+3.4	16.7	-6.6
		+ ALWAYS	44.4	+1.1	43.3	0.0	35.6	+2.3	46.7	+3.4	40.0	+6.7	20.0	-3.3
	GPT-5.3-Codex	NO-SKILL	44.4	-	-	-	34.4	-	36.7	-	46.7	-	20.0	-
		ZERO-SHOT	46.7	+2.3	-	-	34.4	0.0	43.3	+6.6	40.0	-6.7	20.0	0.0
		EXPERIENCE	42.2	-2.2	48.9	+4.5	31.1	-3.3	40.0	+3.3	40.0	-6.7	13.3	-6.7
		+ ALWAYS	44.4	0.0	48.9	+4.5	34.4	0.0	36.7	0.0	43.3	-3.4	23.3	+3.3
	GPT-5.2-Codex	NO-SKILL	43.3	-	-	-	36.7	-	40.0	-	53.3	-	16.7	-
		ZERO-SHOT	45.6	+2.3	-	-	30.0	-6.7	33.3	-6.7	36.7	-16.6	20.0	+3.3
		EXPERIENCE	45.6	+2.3	42.2	-1.1	34.4	-2.3	36.7	-3.3	43.3	-10.0	23.3	+6.6
		+ ALWAYS	46.7	+3.4	46.7	+3.4	38.9	+2.2	50.0	+10.0	43.3	-10.0	23.3	+6.6
GEMINI CLI	Gemini 3.1 Pro	NO-SKILL	40.0	-	-	-	35.6	-	40.0	-	46.7	-	20.0	-
		ZERO-SHOT	43.3	+3.3	-	-	36.7	+1.1	36.7	-3.3	46.7	0.0	26.7	+6.7
		EXPERIENCE	47.8	+7.8	53.3	+13.3	32.2	-3.4	36.7	-3.3	36.7	-10.0	23.3	+3.3
		+ ALWAYS	42.2	+2.2	41.1	+1.1	33.3	-2.3	33.3	-6.7	43.3	-3.4	23.3	+3.3
	Gemini 3 Flash	NO-SKILL	40.0	-	-	-	35.6	-	30.0	-	53.3	-	23.3	-
		ZERO-SHOT	38.9	-1.1	-	-	40.0	+4.4	43.3	+13.3	53.3	0.0	23.3	0.0
		EXPERIENCE	40.0	0.0	43.3	+3.3	33.3	-2.3	36.7	+6.7	40.0	-13.3	23.3	0.0
		+ ALWAYS	35.6	-4.4	40.0	0.0	36.7	+1.1	36.7	+6.7	43.3	-10.0	30.0	+6.7
	Gemini 2.5 Pro	NO-SKILL	30.0	-	-	-	26.7	-	26.7	-	30.0	-	23.3	-
		ZERO-SHOT	28.9	-1.1	-	-	15.6	-11.1	13.3	-13.4	13.3	-16.7	20.0	-3.3
		EXPERIENCE	27.8	-2.2	32.2	+2.2	20.0	-6.7	20.0	-6.7	30.0	0.0	10.0	-13.3
		+ ALWAYS	31.1	+1.1	34.4	+4.4	25.6	-1.1	40.0	+13.3	20.0	-10.0	16.7	-6.6

skills, but may not create, revise, retire, or otherwise modify the library. This phase measures whether prior skill evolution transfers to harder tasks without allowing adaptation on the evaluation instance itself. When replay is enabled, we rerun the original acquisition tasks using the final frozen library. Replay does not update the library. It provides a within-environment counterfactual: the same learning tasks are solved once before the relevant skills have matured and once after the library has evolved.

4.5 Scoring

For each task attempt a , the verifier returns an outcome score O_a , a process score P_a , an overall score G_a , and a binary success indicator B_a . Outcome measures functional correctness through public and hidden tests, while process measures whether the agent followed the intended procedure. For any attempt set $\mathcal{I}$, we report

$$\text{SR}(\mathcal{I}) = \frac{1}{|\mathcal{I}|} \sum_{a \in \mathcal{I}} B_a. \quad (6)$$

We use LSR, RSR, and ESR for acquisition, replay, and frozen deployment success rates, respectively. Deployment is further broken down into CSSR, ARSR, and CompSR over context-shift, adversarial, and composition attempts.

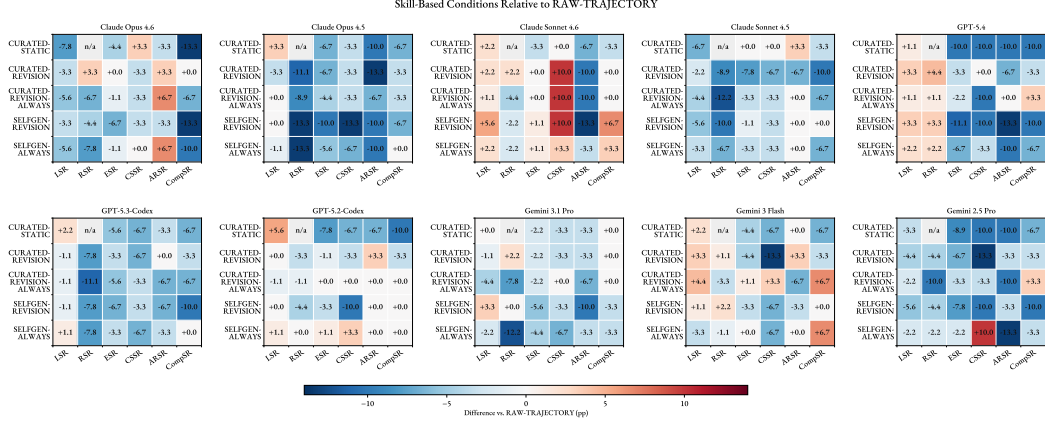


Figure 4: **Skill-based conditions relative to RAW-TRAJECTORY.** Each panel shows one model, with rows denoting skill variants and columns denoting success metrics. Cells report percentage-point differences from the corresponding RAW-TRAJECTORY baseline. Red indicates improvement, blue indicates degradation, and gray indicates unavailable comparisons.

5 Experimental Setup and Results

5.1 Agent Harnesses and Models

We evaluate SKILLEVBENCH with three agent harnesses: Claude Code [35], Codex CLI [36], and Gemini CLI [37]. We run all harnesses under the same benchmark protocol. We test ten model configurations across the three harnesses. Claude Code is evaluated with Opus 4.6, Opus 4.5, Sonnet 4.6, and Sonnet 4.5. Codex CLI is evaluated with GPT-5.4, GPT-5.3-Codex, and GPT-5.2-Codex. Gemini CLI is evaluated with Gemini 3.1 Pro, Gemini 3 Flash, and Gemini 2.5 Pro.

5.2 Experiment Variants

We evaluate eight primary variants. NO-SKILL uses no persistent memory. RAW-TRAJECTORY retrieves compacted same-family acquisition trajectories, without inducing procedural skills. CURATED-STATIC provides fixed curated skills. CURATED-REVISION and CURATED-REVISION-ALWAYS start from curated skills and revise them after failed or all acquisition attempts, respectively. SELFGEN-ZERO-SHOT generates fixed metadata-only skills before the *canonical* task. SELFGEN-REVISION induces skills from *canonical* trajectories and revises after failed later acquisition attempts. SELFGEN-ALWAYS updates after every acquisition attempt. More experiment results are presented in Appendix Sec. D.

5.3 Main Results and Findings

Overall observation. Tables 1 and 2 compare skill-based conditions against NO-SKILL, while Figure 4 compares the same skill-based conditions against RAW-TRAJECTORY. Together, they show that current agents exhibit local procedural adaptation but not reliable reusable skill formation. Skill-based conditions can improve acquisition or replay, and some model-condition pairs achieve strong gains on specific deployment axes. However, these gains do not consistently transfer across ESR, CSSR, ARSR, and CompSR. The RAW-TRAJECTORY comparison further suggests a lossy abstraction bottleneck: agents often use episodic traces more effectively than the distilled skills derived from them.

Local gains do not imply reusable skill formation. Both CURATED-START and SELF-GENERATED settings show cases where skills improve LSR or RSR but fail on deployment metrics. For example, under SELF-GENERATED EXPERIENCE, Claude Opus 4.6 improves LSR by 5.5 pp and RSR by 10.0 pp relative to NO-SKILL, but decreases ESR, CSSR, and CompSR. Claude Sonnet 4.6 under the same condition improves LSR, RSR, CSSR, and CompSR, yet drops by 10.0 pp on ARSR. Similarly, GPT-5.2-Codex with SELF-GENERATED + ALWAYS improves most metrics but

remains 10.0 pp below NO-SKILL on ARSR. These examples suggest that replay recovery and local improvement can reflect useful patches without guaranteeing robust procedural reuse.

Deployment metrics expose distinct failure modes. The deployment metrics separate transfer, implicit invocation, shortcut resistance, and composition. This separation matters because failures appear on different axes. GPT-5.4 under CURATED-START + ALWAYS improves ESR, ARSR, and CompSR, but decreases CSSR, suggesting an implicit-invocation failure when the skill need is embedded in a broader context. Gemini 3 Flash under the same condition gains strongly on CSSR and improves CompSR, yet drops sharply on ARSR, showing shortcut vulnerability. GPT-5.4 under SELF-GENERATED EXPERIENCE improves LSR, RSR, and ARSR, but loses on CompSR, indicating weak modularity. Thus, a single success rate would hide whether a skill fails through over-specialized patching, missed invocation, shortcut reliance, or weak composition.

Raw trajectories reveal a lossy abstraction bottleneck. Figure 4 provides the clearest evidence that abstraction from episodes into skills remains unreliable. If skill abstraction preserved the reusable procedure, skill-based conditions should match or exceed RAW-TRAJECTORY. Instead, the heatmap is predominantly negative across models, variants, and metrics. For example, GPT-5.4 under SELF-GENERATED EXPERIENCE exceeds RAW-TRAJECTORY on LSR and RSR, but falls substantially on ESR, CSSR, ARSR, and CompSR. This suggests that distilled skills may recover behavior near the original episode while losing contextual and procedural cues needed for transfer, robustness, and composition.

More skill updates are not monotonically better. The + ALWAYS policy reveals a coverage–drift trade-off. In the SELF-GENERATED setting, more frequent updates can improve deployment-side coverage: GPT-5.2-Codex improves from EXPERIENCE to + ALWAYS on ESR and CSSR, and Gemini 2.5 Pro also gains on several deployment metrics. However, always updating can reduce local gains, as Gemini 3.1 Pro drops from strong LSR and RSR improvements under EXPERIENCE to much smaller gains under + ALWAYS. In the CURATED-START setting, + ALWAYS also does not dominate + REVISION. Thus, skill evolution is not simply a matter of writing or revising more often; agents need update policies that preserve generality rather than accumulating episode-specific detail.

6 Conclusion

We introduced SKILLEVOLBENCH, a diagnostic benchmark for evaluating whether agents can transform episodic task experience into reusable procedural skills. Rather than only measuring whether skills help at inference time, SKILLEVOLBENCH targets the missing step from experience reuse to skill formation. Its role-conditioned task families, verifier-backed feedback, frozen deployment phase, replay setting, and RAW-TRAJECTORY control help separate local task recovery from transferable procedural reuse. Our experiments show that current agentic LLMs often adapt locally but rarely form reliable reusable skills. Skill-based conditions can improve acquisition or replay, yet their gains are unstable under transfer, context shift, adversarial variants, and composition. The comparison with RAW-TRAJECTORY further reveals a lossy abstraction bottleneck: agents often use episodic traces more effectively than the distilled skills derived from them. These results suggest that the key challenge is not simply storing more experience or writing more skills, but preserving the right procedural structure while discarding episode-specific noise.

7 Limitations and Future Directions

SKILLEVOLBENCH is designed as a diagnostic benchmark and therefore does not cover every form of lifelong agent improvement. It focuses on external skill artifacts rather than parameter updates, persistent memory systems, or human-in-the-loop skill curation. Its environments and task families cover diverse procedural patterns, but they still represent a bounded slice of real-world agent behavior. Future skill-evolution systems need mechanisms that decide when an update improves generality, detect over-specialized patches, preserve useful procedural evidence, and support modular composition across tasks. Such mechanisms may make agents more reliable by exposing when learned procedures fail under transfer or adversarial conditions, while also serving as safeguards as more capable agents accumulate reusable procedures over time. SKILLEVOLBENCH provides a testbed for measuring progress on this step from one-off experience to durable procedural knowledge.

References

- [1] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. R. Narasimhan, and Y. Cao, “React: Synergizing reasoning and acting in language models,” in *The Eleventh International Conference on Learning Representations*, 2023.
- [2] T. Schick, J. Dwivedi-Yu, R. Dessì, R. Raileanu, M. Lomeli, E. Hambro, L. Zettlemoyer, N. Cancedda, and T. Scialom, “Toolformer: Language models can teach themselves to use tools,” *Advances in neural information processing systems*, vol. 36, pp. 68 539–68 551, 2023.
- [3] S. Zhou, F. F. Xu, H. Zhu, X. Zhou, R. Lo, A. Sridhar, X. Cheng, T. Ou, Y. Bisk, D. Fried, U. Alon, and G. Neubig, “Webarena: A realistic web environment for building autonomous agents,” in *The Twelfth International Conference on Learning Representations*, 2024.
- [4] C. E. Jimenez, J. Yang, A. Wettig, S. Yao, K. Pei, O. Press, and K. R. Narasimhan, “SWE-bench: Can language models resolve real-world github issues?” in *The Twelfth International Conference on Learning Representations*, 2024.
- [5] N. Shinn, F. Cassano, A. Gopinath, K. Narasimhan, and S. Yao, “Reflexion: Language agents with verbal reinforcement learning,” *Advances in neural information processing systems*, vol. 36, pp. 8634–8652, 2023.
- [6] A. Zhao, D. Huang, Q. Xu, M. Lin, Y.-J. Liu, and G. Huang, “Expel: Llm agents are experiential learners,” in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 38, no. 17, 2024, pp. 19 632–19 642.
- [7] L. Zheng, R. Wang, X. Wang, and B. An, “Synapse: Trajectory-as-exemplar prompting with memory for computer control,” in *The Twelfth International Conference on Learning Representations*, 2024.
- [8] Z. Z. Wang, J. Mao, D. Fried, and G. Neubig, “Agent workflow memory,” in *Forty-second International Conference on Machine Learning*, 2025.
- [9] Agent Skills, “Agent skills specification,” <https://agentskills.io/specification>.
- [10] Anthropic, “Equipping agents for the real world with agent skills,” <https://www.anthropic.com/engineering/equipping-agents-for-the-real-world-with-agent-skills>, Oct. 2025.
- [11] X. Li, W. Chen, Y. Liu, S. Zheng, X. Chen, Y. He, Y. Li, B. You, H. Shen, J. Sun *et al.*, “Skillsbench: Benchmarking how well agent skills work across diverse tasks,” *arXiv preprint arXiv:2602.12670*, 2026.
- [12] H. Trivedi, T. Khot, M. Hartmann, R. Manku, V. Dong, E. Li, S. Gupta, A. Sabharwal, and N. Balasubramanian, “Appworld: A controllable world of apps and people for benchmarking interactive coding agents,” in *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 2024, pp. 16 022–16 076.
- [13] M. A. Merrill, A. G. Shaw, N. Carlini, B. Li, H. Raj, I. Bercovich, L. Shi, J. Y. Shin, T. Walshe, E. K. Buchanan *et al.*, “Terminal-bench: Benchmarking agents on hard, realistic tasks in command line interfaces,” *arXiv preprint arXiv:2601.11868*, 2026.
- [14] X. Deng, Y. Gu, B. Zheng, S. Chen, S. Stevens, B. Wang, H. Sun, and Y. Su, “Mind2web: Towards a generalist agent for the web,” *Advances in Neural Information Processing Systems*, vol. 36, pp. 28 091–28 114, 2023.
- [15] B. Gou, Z. Huang, Y. Ning, Y. Gu, M. Lin, W. Qi, A. Kopanev, B. Yu, B. J. Gutierrez, Y. Shu, C. H. Song, J. Wu, S. Chen, H. N. Moussa, T. ZHANG, J. Xie, Y. Li, T. Xue, Z. Liao, K. Zhang, B. Zheng, Z. Cai, V. Rozgic, M. Ziyadi, H. Sun, and Y. Su, “Mind2web 2: Evaluating agentic search with agent-as-a-judge,” in *The Thirty-ninth Annual Conference on Neural Information Processing Systems Datasets and Benchmarks Track*, 2026.
- [16] T. Xie, D. Zhang, J. Chen, X. Li, S. Zhao, R. Cao, T. J. Hua, Z. Cheng, D. Shin, F. Lei, Y. Liu, Y. Xu, S. Zhou, S. Savarese, C. Xiong, V. Zhong, and T. Yu, “OSWorld: Benchmarking multimodal agents for open-ended tasks in real computer environments,” in *The Thirty-eight Conference on Neural Information Processing Systems Datasets and Benchmarks Track*, 2024.
- [17] S. Yao, N. Shinn, P. Razavi, and K. R. Narasimhan, “ $\{\tau\}$ -bench: A benchmark for $\{\text{T}\}$ ool- $\{\text{A}\}$ gent- $\{\text{U}\}$ ser interaction in real-world domains,” in *The Thirteenth International Conference on Learning Representations*, 2025.
- [18] F. F. Xu, Y. Song, B. Li, Y. Tang, K. Jain, M. Bao, Z. Z. Wang, X. Zhou, Z. Guo, M. Cao, M. Yang, H. Y. Lu, A. Martin, Z. Su, L. M. Maben, R. Mehta, W. Chi, L. K. Jang, Y. Xie, S. Zhou, and G. Neubig, “Theagentcompany: Benchmarking LLM agents on consequential real world tasks,” in *The Thirty-ninth Annual Conference on Neural Information Processing Systems Datasets and Benchmarks Track*, 2026.

- [19] C. Qian, C. Han, Y. Fung, Y. Qin, Z. Liu, and H. Ji, “Creator: Tool creation for disentangling abstract and concrete reasoning of large language models,” in *Findings of the Association for Computational Linguistics: EMNLP 2023*, 2023, pp. 6922–6939.
- [20] T. Cai, X. Wang, T. Ma, X. Chen, and D. Zhou, “Large language models as tool makers,” in *The Twelfth International Conference on Learning Representations*, 2024.
- [21] G. Wang, Y. Xie, Y. Jiang, A. Mandlekar, C. Xiao, Y. Zhu, L. Fan, and A. Anandkumar, “Voyager: An open-ended embodied agent with large language models,” *Transactions on Machine Learning Research*, 2024.
- [22] P. Xia, J. Chen, H. Wang, J. Liu, K. Zeng, Y. Wang, S. Han, Y. Zhou, X. Zhao, H. Chen *et al.*, “Skillrl: Evolving agents via recursive skill-augmented reinforcement learning,” *arXiv preprint arXiv:2602.08234*, 2026.
- [23] Y. Yang, J. Li, Q. Pan, B. Zhan, Y. Cai, L. Du, J. Zhou, K. Chen, Q. Chen, X. Li *et al.*, “Autoskill: Experience-driven lifelong learning via skill self-evolution,” *arXiv preprint arXiv:2603.01145*, 2026.
- [24] H. Zhang, Q. Long, J. Bao, T. Feng, W. Zhang, H. Yue, and W. Wang, “Memskill: Learning and evolving memory skills for self-evolving agents,” *arXiv preprint arXiv:2602.02474*, 2026.
- [25] H. Zhou, S. Guo, A. Liu, Z. Yu, Z. Gong, B. Zhao, Z. Chen, M. Zhang, Y. Chen, J. Li *et al.*, “Memento-skills: Let agents design agents,” *arXiv preprint arXiv:2603.18743*, 2026.
- [26] H. Zhang, S. Fan, H. P. Zou, Y. Chen, Z. Wang, J. Zhou, C. Li, W.-C. Huang, Y. Yao, K. Zheng *et al.*, “Evoskills: Self-evolving agent skills via co-evolutionary verification,” *arXiv preprint arXiv:2604.01687*, 2026.
- [27] Z. Ma, S. Yang, Y. Ji, X. Wang, Y. Wang, Y. Hu, T. Huang, and X. Chu, “Skillclaw: Let skills evolve collectively with agentic evolver,” *arXiv preprint arXiv:2604.08377*, 2026.
- [28] S. Chen, J. Gai, R. Zhou, J. Zhang, T. Zhu, J. Li, K. Wang, Z. Wang, Z. Chen, K. Kaleb *et al.*, “Skillcraft: Can llm agents learn to use tools skillfully?” *arXiv preprint arXiv:2603.00718*, 2026.
- [29] S. Alzubi, N. Provenzano, J. Bingham, W. Chen, and T. Vu, “Evoskill: Automated skill discovery for multi-agent systems,” *arXiv preprint arXiv:2603.02766*, 2026.
- [30] C. Wang, Z. Yu, X. Xie, W. Yao, R. Fang, S. Qiao, K. Cao, G. Zheng, X. Qi, P. Zhang *et al.*, “Skillx: Automatically constructing skill knowledge bases for agents,” *arXiv preprint arXiv:2604.04804*, 2026.
- [31] T. Han, Y. Zhang, W. Song, C. Fang, Z. Chen, Y. Sun, and L. Hu, “Swe-skills-bench: Do agent skills actually help in real-world software engineering?” *arXiv preprint arXiv:2603.15401*, 2026.
- [32] Kilo AI, “Pinchbench: Real-world benchmarks for ai coding agents,” <https://github.com/pinchbench/skill>, 2026.
- [33] Anthropic, “Skills: Public repository for agent skills,” <https://github.com/anthropics/skills>, 2025.
- [34] —, “Skill creator,” <https://claude.com/plugins/skill-creator>, 2026.
- [35] —, “Claude code overview,” <https://code.claude.com/docs/en/overview>, 2026.
- [36] OpenAI, “Codex cli,” <https://developers.openai.com/codex/cli>, 2026.
- [37] Google Cloud, “Gemini cli,” <https://docs.cloud.google.com/gemini/docs/codeassist/gemini-cli>, 2026.

373 A Complete Family Catalog

374 This appendix catalogs the full set of environment-level skill families in SKILLEVOLBENCH. Each
 375 family corresponds to one procedural skill and contains six role-instantiated tasks.

Table 3: Complete skill-family catalog. Descriptions are taken from the benchmark family metadata.

Family	Skill	Environment	Family Description
E1-LS1	systematic-error-diagnosis	E1: Code Debugging Modification	Systematically diagnose and fix runtime errors in multi-file Python and JavaScript/TypeScript projects. Use this skill when a task includes a traceback, stack trace, failing command, failing test, or clear crash across 3-7 source files and you need to trace the root cause through the call chain. Focus on failures with explicit error output or crashes. Trigger on requests like "fix the bug", "resolve the traceback", "debug this project", or "make the failing test pass" when the failure already points to a concrete runtime error.
E1-LS2	dependency-conflict-resolution	E1: Code Debugging Modification	Diagnose and resolve Python package version conflicts in pip-based projects. Use this skill when 'pip install' fails with resolver errors, when 'pip check' reports incompatible versions, or when a Python project's dependency graph contains conflicting version constraints that must be reconciled in 'requirements.txt' or 'pyproject.toml'. Focus on single-project pip workflows with explicit version conflicts and shared dependencies.
E1-LS3	safe-refactoring	E1: Code Debugging Modification	Safely restructure Python code to improve clarity, reduce duplication, and increase maintainability without changing external behavior. Use this skill when the user asks to refactor, clean up, simplify, rename, extract functions or classes, remove dead code, or reorganize Python code while keeping the existing behavior intact. Focus on Python projects with an existing test suite and structural changes only.
E1-LS4	multi-file-bug-fix	E1: Code Debugging Modification	Trace and fix bugs that span multiple files in Python and JavaScript projects. Use this skill when the visible symptom appears in one file but the root cause is likely upstream or downstream in another file, and the fix requires coordinated updates across several files. Trigger when a user reports wrong output, a broken flow, a changed function contract, a module move, or any bug where one local edit is unlikely to be enough. Focus on projects with roughly 4-7 files involved in the debugging path.
E1-LS5	merge-conflict-resolution	E1: Code Debugging Modification	Resolve git merge conflicts in Python and JavaScript code by combining changes from two branches correctly. Use this skill when 'git merge' reports conflicted files, when a file contains conflict markers like '«««<', '=====', and '»»»>', or when the user asks for help merging two branches without discarding either side's important changes. Focus on conflicts that git reports explicitly in source files with conflict markers.

Family	Skill	Environment	Family Description
E2-LS1	pre-call-parameter-validation	E2: Multi Step Tool Api Orchestration	Validate REST API request parameters locally before sending the request. Use this skill when parameters come from user input, config, or upstream code and an invalid request would waste quota or fail predictably. Focus on single-request validation for required fields, types, ranges, enums, and string formats before making the API call.
E2-LS2	retry-and-backoff	E2: Multi Step Tool Api Orchestration	Retry synchronous REST API calls when server errors occur. Use this skill when an HTTP request may temporarily fail with a 5xx response and the caller needs a simple retry loop with a fixed delay between attempts. Focus on straightforward retry handling for synchronous API calls with status-code-based decision rules.
E2-LS3	pagination-complete-retrieval	E2: Multi Step Tool Api Orchestration	Retrieve the complete dataset from offset-based paginated REST APIs by fetching every page in sequence. Use this skill when an API returns results in numbered pages such as 'page=1&per_page=25' and the user needs all records, not just the first page. Focus on page-number plus page-size pagination with JSON results that can be combined into one list.
E2-LS4	multi-step-orchestration	E2: Multi Step Tool Api Orchestration	Chain two API calls into a simple workflow where the output of the first call is used as input for the second call. Use this skill when a request pattern looks like authenticate then fetch, look up an ID then retrieve details, or any other two-step sequence where Call B depends directly on a value extracted from Call A. Focus on simple two-step API chains only.
E2-LS5	response-validation-fallback	E2: Multi Step Tool Api Orchestration	Validate REST API responses before using the data, and fall back to a secondary source when validation fails. Use this skill when an HTTP response must pass top-level checks for status code, content type, JSON parsing, and required top-level fields before the payload is trusted. Focus on simple response validation plus optional fallback handling.
E3-LS1	schema-inspection-before-query	E3: Data Processing Structured Query	Inspect CSV files and SQL tables before writing queries or data-processing code. Use this skill whenever the user asks for filtering, aggregation, joins, or analysis on tabular data and the schema must be checked first. Focus on quick practical inspection of columns, types, shape, sample rows, and join-key presence before any query is written.
E3-LS2	type-normalization-before-sort	E3: Data Processing Structured Query	Normalize inconsistent pandas column types before sorting or comparing values. Use this skill when a dataframe column mixes date formats, stores numbers as strings, or represents booleans in multiple textual forms and must be standardized before order-dependent operations. Focus on date, numeric, and boolean normalization in pandas DataFrames.
E3-LS3	key-alignment-before-merge	E3: Data Processing Structured Query	Align join keys across data sources before merging. Use this skill when two tables or DataFrames need to be joined but the key columns may differ in name, dtype, or string casing. Focus on simple key alignment before merge - column name matching, type matching, case normalization, and basic merge-result validation.

Family	Skill	Environment	Family Description
E3-LS4	null-safe-filtering-aggregation	E3: Data Processing Structured Query	Handle missing values correctly during data filtering and aggregation in pandas. Use this skill when a DataFrame contains NaN values and the task involves filtering rows, filling missing values, or computing aggregates such as mean, sum, or count. Focus on pandas NaN handling with <code>isna()</code> , <code>dropna()</code> , <code>fillna()</code> , and built-in aggregate functions before drawing conclusions from filtered or aggregated data.
E3-LS5	result-sanity-reconciliation	E3: Data Processing Structured Query	Verify processed tabular results with basic sanity checks before using or returning them. Use this skill after a pandas or SQL data-processing pipeline when you need to confirm that the final output has a reasonable row count, contains the expected columns, looks plausible in sample rows, and does not contain unexpected NaN values in critical fields. Focus on simple final-output verification with <code>df.shape</code> , <code>df.columns</code> , <code>df.describe()</code> , and <code>df.isna().sum()</code> .
E4-LS1	structured-field-extraction	E4: Document Parsing Extraction Transformation	Extract structured data from semi-structured documents such as invoices, forms, and reports, then convert the extracted fields to JSON. Use this skill when a document contains label-value pairs, markdown-style tables, or section headers that organize fields into simple categories. Focus on flat key-value extraction from single-format documents, with basic field validation after extraction.
E4-LS2	cross-format-migration	E4: Document Parsing Extraction Transformation	Convert flat documents from one format to another while preserving source content. Use this skill when the task is to migrate content such as Markdown, CSV, or simple key-value text into JSON, and the goal is to keep all body content represented in the target format. Focus on flat-format migration with explicit source-to-target mapping, parser-based conversion, and a final source-versus-target verification step.
E4-LS3	template-fill-from-context	E4: Document Parsing Extraction Transformation	Fill a template document using information found in one or two context documents. Use this skill when a template contains blank fields or placeholders and the task is to locate matching information in surrounding context, insert it into the template, and return a completed version. Focus on straightforward fill-from-context work with one template, one or two context documents, simple field-name matching, and basic output review.
E4-LS4	document-diff-comparison	E4: Document Parsing Extraction Transformation	Compare two versions of a plain text or markdown document and produce a clear change report. Use this skill when the task is to load two document versions, identify additions, deletions, and modifications, and report each change with its location, original text, and new text. Focus on text-level diff comparison for plain text and markdown documents using basic line-by-line or paragraph-by-paragraph comparison.

Family	Skill	Environment	Family Description
E4-LS5	multi-source-merge-reconciliation	E4: Document Parsing Extraction Transformation	Merge flat data from 2-3 document sources into a single master document using exact key matching. Use this skill when records from multiple sources need to be combined by a shared identifier, duplicate entities need to be unified, conflicting field values need to be flagged, and a source-priority rule should decide which value is kept in the merged output. Focus on exact-key matching across 2-3 sources with flat data structures.
E5-LS1	multi-source-search-filter	E5: Research Information Synthesis	Search and filter the most relevant documents from a collection of 10-20 sources on a given topic. Use this skill when the task is to quickly scan a set of articles, papers, reports, or similar documents, score them for topical relevance, select the top 5 candidates, and rank those selected sources from most to least relevant. Focus on relevance-based selection using keyword matching, topic fit, specific data/examples, and source credibility.
E5-LS2	evidence-grounded-comparison	E5: Research Information Synthesis	Compare 3-5 options using evidence found in provided source documents. Use this skill when the task is to define comparison dimensions, search source material for evidence on each option, build a comparison matrix with source citations, identify which option is strongest on each dimension, and write a summary recommendation based on the completed matrix. Focus on evidence-based comparison tables built from provided documents.
E5-LS3	citation-verification	E5: Research Information Synthesis	Verify that citations in a document accurately represent their sources. Use this skill when a paper, review article, report, or essay makes claims about external sources and the task is to check whether those cited sources actually support the attributed statements. Focus on verifying that claims, quotes, and numerical assertions are supported by the cited source content.
E5-LS4	constrained-summarization	E5: Research Information Synthesis	Generate summaries that must satisfy explicit constraints such as a word limit, required key points, and a requested presentation format. Use this skill when the task is to condense a long document into a shorter summary while staying within a stated word cap, ensuring required points are included, and checking the final draft against those constraints before returning it. Focus on word-limited summaries with required key point coverage.
E5-LS5	contradiction-detection	E5: Research Information Synthesis	Find direct factual contradictions between multiple information sources. Use this skill when two or more documents make claims about the same facts and the task is to extract those claims, group them by topic, compare the stated values, and report any contradictions with their sources. Focus on direct factual contradictions such as different numbers, dates, names, or statistics for the same fact.

Family	Skill	Environment	Family Description
E6-LS1	inbox-triage-prioritization	E6: Communication Scheduling Operations	Classify and prioritize incoming emails using simple category labels and explicit urgency markers. Use this skill when the task is to scan a batch of emails, assign each one to a fixed category, detect clear urgency keywords in the subject or opening sentences, and output a sorted list with High priority first, then Medium, then Low. Focus on keyword-based classification and explicit urgency markers.
E6-LS2	context-aware-reply-drafting	E6: Communication Scheduling Operations	Draft professional reply emails that reference the prior context of a business email thread. Use this skill when a user provides a thread of 3-5 messages and wants a reply that reads the conversation history, identifies the latest question or request, answers it directly, references relevant earlier context when helpful, and matches the tone of the original sender. Focus on concise professional reply drafting for standard business email threads.
E6-LS3	follow-up-tracking	E6: Communication Scheduling Operations	Identify email threads that are overdue for a response by checking the direction of the last message, whether it contains a question or request, and how many business days have passed since it was received. Use this skill when the task is to scan one or more email threads, find items waiting on your reply, and produce an overdue follow-up list sorted by age. Focus on basic overdue detection based on last-message date and direction.
E6-LS4	meeting-scheduling-constraints	E6: Communication Scheduling Operations	Find a common meeting time for 2-3 participants across 2 time zones using fixed UTC offsets. Use this skill when the task is to list participants and time zones, convert business-hour availability into UTC, find the overlapping UTC range, convert that overlap back into local times, and suggest a specific meeting time within the overlap. Focus on scheduling with fixed UTC offsets and standard business hours.
E6-LS5	thread-action-extraction	E6: Communication Scheduling Operations	Extract explicit action items from formal business email threads. Use this skill when the task is to read an email conversation, identify direct requests, named assignments, and explicit sender commitments, and output structured action items with a task description, assignee, deadline, and source email. Focus on extracting explicit action items from professional email threads.

B Task Design Catalog

This appendix lists all task instances. Design signals record gap exposure, trap annotations, and composition requirements from benchmark metadata; task instruction bodies are omitted.

B.1 E1: Code Debugging & Modification

Table 4: Task design catalog for E1: Code Debugging & Modification.

Task	Family	Role / Phase	Title	Design Signals
E1-LS1-T1	E1-LS1	canonical / learning	Fix the intermittent 500s in our analytics API	gap=none
E1-LS1-T2	E1-LS1	enriched / learning	Close the auth bypass in the FinGuard API	gap=gap1-no-crash-behavior-error
E1-LS1-T3	E1-LS1	variant / learning	Fix the report job so timeouts are not silently treated as real zeros	gap=gap2-async-silent-failure

Task	Family	Role / Phase	Title	Design Signals
E1-LS1-T4	E1-LS1	context-shift / evaluation	Finish the webhook work in this repo	gap=implicit-precondition-debugging
E1-LS1-T5	E1-LS1	adversarial / evaluation	Fix the tax calculation bug without patching around individual states	gap=shortcut-trap; traps=shortcut, generic_trap
E1-LS1-T6	E1-LS1	composition / evaluation	Fix the blank profile page by tracing the full failure path	gap=multi-hop-error-cascade; composition=cross_family; requires E1-LS1.systematic-error-diagnosis, E1-LS3.safe-refactoring, E1-LS4.multi-file-bug-fix
E1-LS2-T1	E1-LS2	canonical / learning	Fix the runtime dependency issue in this Python project	gap=none
E1-LS2-T2	E1-LS2	enriched / learning	Fix the React 18 dependency conflict in this frontend app	gap=gap1-npm-peer-dependency
E1-LS2-T3	E1-LS2	variant / learning	Fix the Poetry dependency conflict in this monorepo	gap=gap2-monorepo-cross-package
E1-LS2-T4	E1-LS2	context-shift / evaluation	Finish the pending PR for CSV export	gap=implicit-blocker-in-context
E1-LS2-T5	E1-LS2	adversarial / evaluation	Fix the ML pipeline crash after the dependency update	gap=shortcut-pin-old-version-trap; traps=shortcut, generic_trap
E1-LS2-T6	E1-LS2	composition / evaluation	Upgrade Alembic and make the SQLAlchemy stack compatible	gap=multi-layer-upgrade-migration; composition=cross_family; requires E1-LS2.dependency-conflict-resolution, E1-LS4.multi-file-bug-fix
E1-LS3-T1	E1-LS3	canonical / learning	Reduce duplicated validation in the intake handlers	gap=none
E1-LS3-T2	E1-LS3	enriched / learning	Replace the event switch with a strategy mapping without changing behavior	gap=gap1-fallthrough-sideeffects
E1-LS3-T3	E1-LS3	variant / learning	Refactor the processor hierarchy into composition without breaking TypeScript constraints	gap=gap2-typesystem-super-chain
E1-LS3-T4	E1-LS3	context-shift / evaluation	Add plugin support, but do it in a maintainable way	gap=implicit-refactor-before-feature
E1-LS3-T5	E1-LS3	adversarial / evaluation	Refactor the parse/transform/format chain without collapsing empty-input behavior	gap=gap1-boundary-behavior-preservation; traps=gap1-boundary-behavior-preservation
E1-LS3-T6	E1-LS3	composition / evaluation	Refactor the analysis function and use coverage to close the gaps	gap=coverage-driven-edge-case-diagnosis; composition=cross_family; requires E1-LS3.safe-refactoring, E1-LS4.multi-file-bug-fix
E1-LS4-T1	E1-LS4	canonical / learning	Fix the false-positive change detection in the ETL pipeline	gap=none
E1-LS4-T2	E1-LS4	enriched / learning	Fix the blank GraphQL profile fields	gap=gap1-cross-system-naming
E1-LS4-T3	E1-LS4	variant / learning	Fix the false mismatches between the two services	gap=gap2-json-precision-loss
E1-LS4-T4	E1-LS4	context-shift / evaluation	Run the migration and make sure the timeline stays correct	gap=implicit-timezone-bug-in-migration
E1-LS4-T5	E1-LS4	adversarial / evaluation	Fix the profile page error handling end to end	gap=frontend-workaround-trap; traps=generic_trap
E1-LS4-T6	E1-LS4	composition / evaluation	Fix the order precision issue, then lock it down with tests and docs	gap=precision-fix-plus-tests-plus-docs; composition=cross_family; requires E1-LS1.systematic-error-diagnosis, E1-LS4.multi-file-bug-fix
E1-LS5-T1	E1-LS5	canonical / learning	Task: Resolve the Merge Before We Cut the Utility Rollout	gap=none
E1-LS5-T2	E1-LS5	enriched / learning	Task: Review a "Successful" Merge That Still Broke Validation	gap=gap1-semantic-conflict-after-clean-merge
E1-LS5-T3	E1-LS5	variant / learning	Task: Finish the Config Merge Without Breaking the YAML Structure	gap=gap2-format-sensitive-config-merge
E1-LS5-T4	E1-LS5	context-shift / evaluation	Task: Unblock the v2.1 Release Merge	gap=implicit-conflict-during-release-workflow
E1-LS5-T5	E1-LS5	adversarial / evaluation	Task: Resolve the Database Merge Without Dropping the Safer Query Changes	gap=shortcut-trap-accept-ours-drops-security-fix; traps=shortcut, generic_trap
E1-LS5-T6	E1-LS5	composition / evaluation	Task: Finish the End-of-Sprint Feature Merge	gap=merge-plus-regression-diagnosis; composition=cross_family; requires E1-LS1.systematic-error-diagnosis, E1-LS4.multi-file-bug-fix, E1-LS5.merge-conflict-resolution

Table 5: Task design catalog for E2: Tool & API Orchestration.

Task	Family	Role / Phase	Title	Design Signals
E2-LS1-T1	E2-LS1	canonical / learning	Task: Process the Weather Batch Without Sending Bad Requests	gap=none
E2-LS1-T2	E2-LS1	enriched / learning	Task: Validate Date Ranges Before Sending Search Requests	gap=gap1-cross-parameter-constraint-with-timezone-normalization
E2-LS1-T3	E2-LS1	variant / learning	Task: Import the Batch and Reject Only the Bad Rows	gap=gap2-batch-partial-reject-with-per-row-reporting
E2-LS1-T4	E2-LS1	context-shift / evaluation	Task: Turn the Sales Query Into a Valid Search Request	gap=implicit-validation-inside-search-pipeline
E2-LS1-T5	E2-LS1	adversarial / evaluation	Task: Stop Depending on Backend 400 Payloads	gap=remote-400-payload-trap; traps=generic_trap
E2-LS1-T6	E2-LS1	composition / evaluation	Task: Complete the Validation and Enrichment Pipeline Before Calling Transactions	gap=validate-enrich-normalize-call-composition; composition=cross_family; requires E2-LS1.pre-call-parameter-validation, E2-LS5.response-validation-fallback
E2-LS2-T1	E2-LS2	canonical / learning	Task: Respect Retry-After When the API Rate Limits Us	gap=none
E2-LS2-T2	E2-LS2	enriched / learning	Task: Make the 503 Retry Strategy Production-Safe	gap=gap1-exponential-backoff-and-jitter-for-503
E2-LS2-T3	E2-LS2	variant / learning	Task: Retry the Order Create Call Without Duplicating Orders	gap=gap2-non-idempotent-post-retry-requires-shared-idempotency-key
E2-LS2-T4	E2-LS2	context-shift / evaluation	Task: Retry Only the Failing Step in the Sync Pipeline	gap=implicit-precise-mid-pipeline-retry
E2-LS2-T5	E2-LS2	adversarial / evaluation	Task: Stop Blind Retries From Hiding Token Expiry	gap=trap-401-needs-refresh-not-blind-retry; traps=generic_trap
E2-LS2-T6	E2-LS2	composition / evaluation	Task: Add a Real Circuit Breaker to the Downstream Client	gap=composition-retry-and-backoff-with-circuit-breaker-state-management; composition=cross_family; requires E2-LS2.retry-and-backoff, E2-LS4.multi-step-orchestration
E2-LS3-T1	E2-LS3	canonical / learning	Task: Retrieve the Full User Export from the Offset API	gap=none
E2-LS3-T2	E2-LS3	enriched / learning	Task: Fix the Event Stream Pagination Client	gap=gap1-cursor-based-pagination-with-variable-page-size
E2-LS3-T3	E2-LS3	variant / learning	Task: Make the Order Export Consistent While New Rows Arrive	gap=gap2-pagination-during-concurrent-inserts
E2-LS3-T4	E2-LS3	context-shift / evaluation	Task: Get the Full Catalog Export	gap=implicit-pagination-discovery
E2-LS3-T5	E2-LS3	adversarial / evaluation	Task: Retrieve the Full Set of Monthly Reports	gap=trap-trusting-first-total-pages-value; traps=generic_trap
E2-LS3-T6	E2-LS3	composition / evaluation	Task: Merge Both Regional Order Feeds into One Complete Export	gap=composition-pagination-plus-retry-merge-sort; composition=cross_family; requires E2-LS2.retry-and-backoff, E2-LS3.pagination-complete-retrieval
E2-LS4-T1	E2-LS4	canonical / learning	Task: Authenticate, Collect Item Details, and Save the Full Export	gap=none
E2-LS4-T2	E2-LS4	enriched / learning	Task: Process Each User's Offers Through the Correct Branch	gap=gap1-conditional-branch-pipeline
E2-LS4-T3	E2-LS4	variant / learning	Task: Generate the Report Through the Async Job API	gap=gap2-async-polling-wait-pattern
E2-LS4-T4	E2-LS4	context-shift / evaluation	Task: Build the Cross-Regional Sales Summary	gap=implicit-multi-source-aggregation-flow
E2-LS4-T5	E2-LS4	adversarial / evaluation	Task: Fetch the Latest Dataset Through the Configured Endpoint	gap=trap-static-endpoint-assumption; traps=generic_trap
E2-LS4-T6	E2-LS4	composition / evaluation	Task: Run the Full User Workflow End-to-End	gap=composition-orchestration-retry-validation; composition=cross_family; requires E2-LS2.retry-and-backoff, E2-LS4.multi-step-orchestration, E2-LS5.response-validation-fallback
E2-LS5-T1	E2-LS5	canonical / learning	Task: Product Feed Collection	gap=none
E2-LS5-T2	E2-LS5	enriched / learning	Task: Product sync during API rollout	gap=gap1-nested-wrapper-structural-change
E2-LS5-T3	E2-LS5	variant / learning	Task: Pricing feed validation	gap=gap2-semantic-anomaly
E2-LS5-T4	E2-LS5	context-shift / evaluation	Task: Inventory export	gap=implicit-response-validation
E2-LS5-T5	E2-LS5	adversarial / evaluation	Task: User export during field migration	gap=trap-blind-200-trust; traps=generic_trap
E2-LS5-T6	E2-LS5	composition / evaluation	Task: Primary/backup product merge	gap=composition-validation-fallback-merge; composition=cross_family; requires E2-LS1.pre-call-parameter-validation, E2-LS5.response-validation-fallback

381 **B.3 E3: Data Processing & Structured Query**

Table 6: Task design catalog for E3: Data Processing & Structured Query.

Task	Family	Role / Phase	Title	Design Signals
E3-LS1-T1	E3-LS1	canonical / learning	Task: Repair The ERP Sales Export Pipeline	gap=none
E3-LS1-T2	E3-LS1	enriched / learning	Task: Stabilize The German Revenue Export Loader	gap=gap1-invisible-characters-and-encoding
E3-LS1-T3	E3-LS1	variant / learning	Task: Repair The Revenue Normalization Job	gap=gap2-mixed-column-types
E3-LS1-T4	E3-LS1	context-shift / evaluation	Task: Repair The Customer Revenue Join	gap=implicit-schema-check-before-join
E3-LS1-T5	E3-LS1	adversarial / evaluation	Task: Fix The Sales Metric Extractor	gap=shortcut-case-sensitive-columns; traps=shortcut
E3-LS1-T6	E3-LS1	composition / evaluation	Task: Rebuild The Dirty Revenue Validation Pipeline	gap=composition-clean-query-validate; composition=cross_family; requires E3-LS1.schema-inspection-before-query, E3-LS5.result-sanity-reconciliation
E3-LS2-T1	E3-LS2	canonical / learning	Task: Repair The Log Timeline Sorter	gap=none
E3-LS2-T2	E3-LS2	enriched / learning	Task: Repair The German Contact Sort Order	gap=gap1-locale-aware-sorting
E3-LS2-T3	E3-LS2	variant / learning	Task: Repair The Employee Ranking Sort	gap=gap2-stable-multi-key-sort
E3-LS2-T4	E3-LS2	context-shift / evaluation	Task: Build The Q3 Sales Leaderboard	gap=implicit-ranking-sort
E3-LS2-T5	E3-LS2	adversarial / evaluation	Task: Fix The Product Catalog Ordering	gap=string-numeric-natural-sort-trap; traps=generic_trap
E3-LS2-T6	E3-LS2	composition / evaluation	Task: Rebuild The Transaction Ordering Pipeline	gap=composition-sort-dedup-aggregate; composition=cross_family; requires E3-LS2.type-normalization-before-sort, E3-LS3.key-alignment-before-merge, E3-LS4.null-safe-filtering-aggregation
E3-LS3-T1	E3-LS3	canonical / learning	Task: Merge Customer Profiles With Order Summaries	gap=none
E3-LS3-T2	E3-LS3	enriched / learning	Task: Merge Population And GDP By City	gap=gap1-fuzzy-match
E3-LS3-T3	E3-LS3	variant / learning	Task: Calculate Average Course Load Per Student	gap=gap2-many-to-many-fanout
E3-LS3-T4	E3-LS3	context-shift / evaluation	Task: Compare 2023 And 2024 Product Sales	gap=implicit
E3-LS3-T5	E3-LS3	adversarial / evaluation	Task: Merge Users With Transactions For Spending Totals	gap=trap-wrong-key; traps=generic_trap
E3-LS3-T6	E3-LS3	composition / evaluation	Task: Build A Clean Customer Revenue Dataset From Three Sources	gap=composition; composition=cross_family; requires E3-LS2.type-normalization-before-sort, E3-LS3.key-alignment-before-merge
E3-LS4-T1	E3-LS4	canonical / learning	Task: Average Order Amount	gap=none
E3-LS4-T2	E3-LS4	enriched / learning	Task: Regional Sales Totals	gap=gap1-groupby-dropna
E3-LS4-T3	E3-LS4	variant / learning	Task: Employee Statistics	gap=gap2-sentinel-values
E3-LS4-T4	E3-LS4	context-shift / evaluation	Task: Channel Conversion Rates	gap=implicit-null-denominator
E3-LS4-T5	E3-LS4	adversarial / evaluation	Task: Temperature Analysis	gap=trap-fillna-zero; traps=generic_trap
E3-LS4-T6	E3-LS4	composition / evaluation	Task: Unified Product Inventory	gap=composition-multi-source-null; composition=cross_family; requires E3-LS3.key-alignment-before-merge, E3-LS4.null-safe-filtering-aggregation
E3-LS5-T1	E3-LS5	canonical / learning	Task: Electronics Catalog Extract	gap=baseline-rowcount-sanity
E3-LS5-T2	E3-LS5	enriched / learning	Task: Quarterly Revenue Sanity Review	gap=statistical-anomaly-detection
E3-LS5-T3	E3-LS5	variant / learning	Task: Revenue Cross-Check by Product Line	gap=cross-query-consistency
E3-LS5-T4	E3-LS5	context-shift / evaluation	Task: March Sales Report	gap=implicit-completeness-check
E3-LS5-T5	E3-LS5	adversarial / evaluation	Task: Active Customer Count	gap=logic-correctness-trap; traps=generic_trap

Task	Family	Role / Phase	Title	Design Signals
E3-LS5-T6	E3-LS5	composition / evaluation	Task: Operations Quality Report	gap=composition-multi-layer-sanity; composition=cross_family; requires E3-LS1.schema-inspection-before-query, E3-LS2.type-normalization-before-sort, E3-LS5.result-sanity-reconciliation

C Implementation-Specific Experiment Settings

This appendix records implementation-specific settings needed to reproduce the experimental protocol. It complements the main-text description of the variants by specifying how models and provider endpoints are routed, how execution trajectories are compacted before being reused as evidence, and the exact Skill Author prompts used for zero-shot generation, experience-based induction, and revision.

C.1 Model and API Configuration

We separate experimental conditions from model/API routing. The baseline and strategy files specify the memory mechanism, retrieval behavior, and skill authoring policy. Model-specific execution details are loaded from a model preset file in `configs/models/*.yaml`. This keeps the same experimental variant runnable across different providers without changing the benchmark logic. Table 7 summarizes the configuration parameters used for this routing.

There are two LLM call surfaces. The first is the task-solving agent, which is launched inside Harbor through an agent adapter. The second is the host-side LLM interface used by components such as the Skill Author and LLM-based retriever. When a model preset is provided, both surfaces are routed through the same preset-specific provider configuration. API secrets are not stored in configuration files; the configs store only environment-variable names.

The provider-specific routing is explicit in the model preset. OpenAI-family models in our runs are served through Azure OpenAI rather than the public OpenAI endpoint. The task-solving agent uses the Codex adapter, while the endpoint and key are forwarded through the OpenAI-compatible environment variables expected by the adapter:

```

provider: azure_openai
harbor_agent_name: codex
agent_model_name: openai/<azure-deployment-name>

agent_env:
  OPENAI_BASE_URL: "${AZURE_OPENAI_ENDPOINT}"
  OPENAI_API_KEY: "${AZURE_OPENAI_API_KEY}"
  CODEX_MODEL_PROVIDER: azure_openai
  CODEX_PROVIDER_ENV_KEY: OPENAI_API_KEY
  CODEX_WIRE_API: responses
  CODEX_MODEL_VERBOSITY: medium

host_litellm:
  model: openai/<azure-deployment-name>
  api_base_env: AZURE_OPENAI_ENDPOINT
  api_key_env: AZURE_OPENAI_API_KEY

```

Thus the `openai/...` model string denotes the OpenAI-compatible model identifier used by Codex and LiteLLM, while the actual endpoint is the Azure OpenAI resource named by `AZURE_OPENAI_ENDPOINT`. The `agent_model_name` must match the Azure deployment name available in that resource.

Claude-family models are served through Amazon Bedrock. The task-solving agent uses the Claude Code adapter in Bedrock mode. Bedrock requires the AWS cross-region inference-profile identifier as the task-agent model name; the host-side LiteLLM model uses the corresponding `bedrock/...` prefix:

```

427 provider: bedrock
428 harbor_agent_name: claude-code
429 agent_model_name: us.anthropic.<bedrock-inference-profile-id>
430
431 agent_env:
432     CLAUDE_CODE_USE_BEDROCK: "1"
433     ANTHROPIC_DEFAULT_OPUS_MODEL: us.anthropic.<...>      # Opus presets
434     ANTHROPIC_DEFAULT_SONNET_MODEL: us.anthropic.<...>    # Sonnet presets
435
436 host_litellm:
437     model: bedrock/us.anthropic.<bedrock-inference-profile-id>

```

438 For Bedrock runs, the host-side client relies on ambient AWS credentials, such as
439 `AWS_BEARER_TOKEN_BEDROCK` and `AWS_REGION`; no `api_base_env` or `api_key_env` is stored
440 in the model preset. Gemini presets use `provider: gemini` with `harbor_agent_name:`
441 `gemini-cli`, an `agent_model_name` of the form `google/<model>`, and `GEMINI_API_KEY` for
442 host-side LiteLLM calls. Kimi presets use an OpenAI-compatible Bedrock-Mantle endpoint with
443 `provider: bedrock_mantle`, `harbor_agent_name: kimi-cli`, `agent_model_name` of the
444 form `openai/moonshotai.<model>`, and endpoint/key variables `KIMI_BEDROCK_BASE_URL` and
445 `KIMI_BEDROCK_API_KEY`.

446 At launch time, the model preset first resolves `agent_env` against the process environment. It then
447 sets `harbor_agent_name` and `model_name` in the baseline config used by Harbor. For host-side
448 calls, the launcher publishes the resolved `host_litellm` settings through `SEVB_HOST_LITELLM_*`
449 variables. The runtime reads these sentinels before falling back to the run-level `api_base`,
450 `api_key_env_var`, and strategy-level `author_model` defaults.

451 C.2 Trajectory Compaction

452 After each Harbor trial, we persist the verifier result and a compacted view of the agent trajectory.
453 The compactor is deterministic and does not call an LLM. It parses the recorded trajectory, selects
454 a small set of informative events, and renders them as a bounded markdown summary. We use
455 two compaction modes: *rich* compaction for Skill Author evidence, and *rough* compaction for the
456 raw-trajectory control baseline. Table 8 lists the parameters for both modes.

457 The rich compaction is stored as `trajectory_compact`. It preserves agent messages, tool calls,
458 observations, verifier summaries, and available `reasoning_content` when the harness records it.
459 This field is used by experience-based induction and revision. For induction, the Skill Author sees
460 the current canonical-task compacted trajectory. For revision, the strategy constructs a cumulative
461 same-family learning history by concatenating prior learning-role compacted trajectories with the
462 current trial. Deployment and replay trials are not used as authoring evidence.

463 The rough compaction is stored as `trajectory_compact_rough`. It is used only by RAW-
464 TRAJECTORY. Rough compaction intentionally drops `reasoning_content`, because the raw-
465 trajectory baseline should expose episodic execution evidence rather than the agent’s private abstrac-
466 tion. The baseline retrieves same-family learning-role trajectories only, in chronological order, with
467 `trajectory_retrieval_k=3`. Thus canonical tasks have no past trajectories, enriched tasks can
468 see the canonical trace, variant tasks can see canonical and enriched traces, and deployment tasks can
469 see the full canonical–enriched–variant chain.

470 The compactor supports several trajectory formats. It first parses the canonical ATIF-style
471 `trajectory.json` with `steps`; if that is not available, it falls back to common JSON shapes
472 such as `events`, `messages`, `trace`, top-level lists, JSONL logs, and finally plain text. For each
473 selected event, the rendered summary records the source kind, message text, optional reasoning
474 content, up to three tool calls, observations or errors, and the verifier verdict with reward. It also
475 extracts referenced skill paths so later analysis can audit whether the agent actually used mounted
476 skills.

Table 7: **Model and API routing parameters.** These parameters describe how model presets and run configs route task-agent and host-side LLM calls. They are configuration fields rather than reported evaluation metrics.

Parameter	Meaning
<code>provider</code>	Backend family used by a model preset for routing and documentation, e.g., Bedrock, Azure OpenAI, Gemini, or an OpenAI-compatible endpoint.
<code>harbor_agent_name</code>	Agent adapter selected for task-solving trials. This determines which preinstalled CLI or agent wrapper Harbor launches for the benchmark task.
<code>agent_model_name</code>	Provider-qualified model identifier passed to the task-solving agent. For deployment-based providers, this is the deployment or inference-profile name expected by the adapter.
<code>agent_env</code>	Environment variables exported before the task-solving agent is constructed. These variables provide provider-specific runtime settings such as endpoint aliases, API-key variable names, wire protocol selection, or CLI trust flags.
<code>agent_kwargs</code>	Non-secret adapter arguments persisted into the Harbor agent configuration. For Codex-style adapters this may include fields such as <code>base_url</code> , <code>provider</code> , <code>env_key</code> , <code>wire_api</code> , <code>api_version</code> , and <code>verbosity</code> .
<code>model_name</code>	The task-agent model field in the resolved baseline config. When a model preset is used, this value is overwritten by <code>agent_model_name</code> .
<code>api_base</code>	Optional endpoint override in the run-level LLM config. If unset, the provider default or preset-specific endpoint is used.
<code>api_key_env_var</code>	Name of the environment variable from which an API key is read. The key value itself is never serialized into benchmark configs.
<code>host_litellm.model</code>	Model identifier used for host-side LiteLLM calls, including Skill Author calls and LLM-based retrieval when enabled.
<code>host_litellm.api_base_env</code>	Environment variable whose value supplies the host-side LiteLLM API base. This is used for provider endpoints such as Azure OpenAI or other OpenAI-compatible gateways.
<code>host_litellm.api_key_env</code>	Environment variable whose value supplies the host-side LiteLLM API key. For providers that use ambient credentials, this field can be omitted.
<code>SEVB_HOST_LITELLM_MODEL</code>	Runtime sentinel set by the launcher from <code>host_litellm.model</code> . It ensures host-side LLM components use the same preset-selected model routing.
<code>SEVB_HOST_LITELLM_API_BASE</code>	Runtime sentinel containing the resolved host-side API endpoint, when an explicit endpoint is required.
<code>SEVB_HOST_LITELLM_API_KEY</code>	Runtime sentinel containing the resolved host-side API key. This value is read from the configured environment variable at launch time.
<code>author_model</code>	Fallback model for Skill Author calls when no model preset is active. Under preset-based runs, host-side calls are routed through the preset model instead.
<code>author_temperature</code>	Sampling temperature for Skill Author generation.
<code>author_max_tokens</code>	Maximum generation length for Skill Author calls.
<code>judge_model</code>	Backward-compatible strategy field for selector/judge-style workflows. In the chain strategy used by the main runs, the Skill Author path does not rely on a separate judge model.
<code>judge_temperature</code>	Sampling temperature for judge/selector calls when that strategy path is enabled.
<code>judge_max_tokens</code>	Maximum generation length for judge/selector calls when that strategy path is enabled.
<code>default_embed_model</code>	Default embedding model used by embedding-based retrieval if no specialized embedder is provided.
<code>harbor_orchestrator_type</code>	Harbor execution backend. The config supports local and remote orchestrators, but this field changes infrastructure rather than the benchmark protocol.
<code>harbor_n_concurrent_trial</code>	Number of concurrent Harbor trials. The lifelong protocol requires this to be 1, because task order and library state must remain sequential.

Table 8: **Trajectory compactor settings.** Both modes are deterministic. Rich compaction is the evidence source for the Skill Author; rough compaction is the evidence source for the raw-trajectory baseline.

Parameter	Rich Compaction	Rough Compaction
head	Keeps the first 8 events.	Keeps the first 4 events.
tail	Keeps the last 12 events.	Keeps the last 8 events.
max_event_chars	8000.	8000.
max_total_tokens	8000 approximate tokens per trial.	8000 approximate tokens per trial.
max_reasoning_chars	None; preserve recorded reasoning content subject to the per-event cap.	0; drop reasoning content entirely.
max_message_chars	None; preserve agent messages subject to the per-event cap.	None; preserve agent messages subject to the per-event cap.
max_observation_chars	3000.	3000.
max_tool_args_chars	200.	100.
signal events	Always keeps events matching traceback, error/exception/failure markers, /skills/, reward.txt, or verifier output.	Same signal-event rule.

477 C.3 Skill Author Prompts

478 The Skill Author is a host-side LLM call used only for skill creation and revision. The task-solving
479 agent never receives these prompts. The implementation separates each call into a stable system
480 prompt and a trial-specific user prompt. The system prompt contains the reusable skill writing contract,
481 while the user prompt supplies the family id, skill id, verifier feedback, trajectory summary, existing
482 skills, and condition-specific instruction. The exact implementation prompt code is reproduced below.

483 **Prompt Rendering Logic.** The Skill Author uses the same stable system contract across authoring
484 calls, paired with a task-specific user message. Revision prompts receive the revision mode, verifier
485 outcome, pass/fail-specific diagnosis instruction, cumulative same-family learning trajectories, and
486 the current same-family skills that may be edited. Experience-based induction prompts receive the
487 family id, latent skill id, canonical-task verifier feedback, and the compacted canonical trajectory.
488 Zero-shot prompts receive only family metadata and the target skill id, and therefore explicitly
489 prohibit invented task-specific gotchas or Tier-3 resources.

490 **Prompt Templates.** The following source excerpt contains the shared system contract and the
491 exact system/user templates for revision, experience-based induction, and zero-shot generation. The
492 listings are split into smaller blocks to avoid LaTeX memory pressure from one very large code box.

493 Shared skill-writing contract

```

494 # three questions the LLM should keep in mind while authoring:
495 # 1. What is a skill (vs a task note)?
496 # 2. What are the three tiers (Tier 1 / Tier 2 / Tier 3)?
497 # 3. What mindset produces a good skill -- comprehensive, adversarial-aware,
498 #    written for the FUTURE agent rather than the current trace.
499 # Kept short on purpose; the detailed limits + best practices live in
500 # _SPEC_BLOCK below. Both are concatenated into every system prompt.
501 _SKILL_MINDSET = ""
502
503 =====
504 WHAT A SKILL IS, AND WHAT YOU ARE BUILDING
505 =====
506
507 A skill is a *reusable capability package* persisted on disk and loaded into
508 a future agent's context whenever it tackles a task in this domain. It is
509 NOT a one-off note about today's trial -- it must generalize to the whole
510 class of similar tasks, including ones the current trace did not cover.
511
512 ## The three tiers (agentskills.io progressive disclosure)
513
514 Tier 1 -- METADATA: the YAML frontmatter at the top of SKILL.md
515           (`name` + `description`). ~100 tokens, ALWAYS in the agent's
516           context. The description is the entire trigger surface --
517           future tasks RETRIEVE this skill solely on its match against

```



```

518         user prompts. Weak description = the skill never fires.
519
520     Tier 2 -- INSTRUCTIONS: the SKILL.md body (markdown, after the
521         frontmatter). Loaded ONLY when Tier 1 matches. <5000 tokens
522         recommended. This is the workflow, gotchas, decision rules,
523         anti-patterns, and worked examples a future agent will follow.
524
525     Tier 3 -- RESOURCES: optional files in `/scripts/`,
526         `/references/`, `/assets/`. Loaded ON DEMAND when
527         SKILL.md explicitly tells the agent to run / read / copy them.
528     Tier 3 holds executable validators (`scripts/`), bulky
529     reference docs (`references/`), and copy-paste templates
530     (`assets/`) -- material that would bloat Tier 2 if inlined.
531     Uncited Tier-3 files are dead weight and rejected.
532
533 ## Mindset: what makes a good skill
534
535 * **Aim for COMPREHENSIVE coverage.** The trial in front of you is one
536 observation. Design the skill so it also handles the obvious harder
537 variants a future task will throw -- different data shapes, missing or
538 malformed fields, partial failures, cross-cutting edge cases, scaled-up
539 inputs. Encode the GENERAL workflow, not just what worked today.
540
541 * **Anticipate ADVERSARIAL conditions.** Future agents will see tasks
542 designed to expose weaknesses: misleading framing, edge cases that
543 look like the canonical case, traps that punish "blindly follow the
544 workflow" patterns, inputs crafted to make a tempting-but-wrong path
545 look correct. Encode the GUARDRAILS -- validation steps, sanity
546 checks, "do not rely on X when Y" anti-patterns -- that catch these
547 before the agent commits to a wrong answer.
548
549 * **Write for the AGENT in the FUTURE, not for the human reading this
550 trace.** The future agent will encounter cases neither you nor the
551 trace has seen. Ask: what knowledge would let it succeed where it
552 would otherwise fail or guess? Cache that knowledge -- runtime paths,
553 schema names, exact APIs, conventions, gotchas -- so the future
554 agent does not have to RE-DERIVE it.
555
556 * **Bias toward DECISION RULES** ("if X pattern appears, do Y instead
557 of Z"; "before doing W, validate V"; "never call A when B is true").
558 Decision rules generalize. Generic prose advice ("be careful",
559 "consider edge cases") is filler.
560
561 * **Keep it CONCISE.** Comprehensive ≠ verbose. Every token in Tier 2
562 is paid context for every future task. Add only what is non-obvious,
563 reusable, and worth its cost. Move bulky detail to Tier 3.
564
565 * **PROCEDURAL knowledge, not EPISODIC memory.** A skill encodes
566 HOW to handle a CLASS of problems. It does NOT record the specific
567 answers, fixes, or trajectory steps from any single trial. WRITE:
568 transferable failure patterns, reusable workflows, trigger
569 conditions, validation checks, decision rules. DO NOT WRITE: the
570 hard-coded fix for one task instance, a literal answer to one
571 problem, the full log of a single trajectory, "in the FinGuard
572 task we did X". The skill must work for tasks neither you nor the
573 trace has seen.
574
575 The detailed agentskills.io spec follows; treat it as the binding
576 contract for shape (directory layout, frontmatter fields, Tier 3
577 folders, etc.).
578
579 ""
580
581
582 # Distilled from the agentskills.io spec, best-practices, optimizing-descriptions,
583 # and using-scripts pages. Embedded into every prompt so the LLM author has the
584 # full design contract without an external lookup.
585 _SPEC_BLOCK = ""\
586 =====
587 AGENT SKILLS -- SPEC + BEST PRACTICES (READ BEFORE WRITING)
588 =====
589
590 ## 1. Directory layout
591
592 A skill is a directory; minimum content is `SKILL.md`. Optional subdirs are
593 `scripts/`, `references/`, and `assets/`:
594
595 ---
596 my-skill/
597 |-- SKILL.md          # Required: metadata + instructions
598 |-- scripts/         # Optional: executable code

```

```

599 |-- references/      # Optional: documentation
600 |-- assets/        # Optional: templates, resources
601 |-- ...            # Any additional files or directories
602 ---
603
604 Reference these from SKILL.md by RELATIVE path (e.g. `scripts/extract.py`,
605 `references/REFERENCE.md`). Keep file references one level deep.
606
607 ## 2. SKILL.md format = YAML frontmatter + Markdown body
608
609 Frontmatter is the YAML between `---` markers at the very top.
610
611 | Field           | Req | Constraint
612 |                 |     |
613 |-----|-----|-----|-----|-----|-----|
614
615 | `name`          | YES | <=64 chars; [a-z0-9-]; no leading/trailing/consecutive `-`; MUST match folder
616 |   name.         |     |
617 | `description`   | YES | <=1024 chars; non-empty; states WHAT the skill does AND WHEN to use it.
618 |                 |     |
619 | `license`       | no  | License name or path to a bundled license file.
620 |                 |     |
621 | `compatibility` | no  | <=500 chars; environment requirements (product, packages, network, runtime
622 |   version).     |     |
623 | `metadata`      | no  | Free key-value map; pick keys unlikely to collide.
624 |                 |     |
625 | `allowed-tools` | no  | Space-separated pre-approved tools (experimental).
626 |                 |     |
627
628 Minimal valid SKILL.md:
629 ---
630 ---
631
632 name: pdf-processing
633 description: Extract PDF text, fill forms, merge files. Use when handling PDFs.
634 ---
635 # PDF processing
636 ... body ...
637 ---
638
639 The Markdown body has no enforced format. Recommended sections: step-by-step
640 instructions, examples, common edge cases. The full body loads into the agent
641 once the skill activates -- treat every token as paid context.
642
643 ## 3. Progressive disclosure (THREE TIERS)
644
645 Skills load in three tiers; design for it.
646
647 Tier 1 -- METADATA (<100 tokens, ALWAYS in context): the YAML frontmatter
648 (`name` + `description`). Used by the agent's matcher to decide
649 whether to load the body. A weak description = the skill never
650 triggers.
651 Tier 2 -- INSTRUCTIONS (<5000 tokens recommended, loaded WHEN ACTIVATED):
652 the SKILL.md body. Keep it under 500 lines. Move bulky detail to
653 Tier 3.
654 Tier 3 -- RESOURCES (loaded ON DEMAND): files in `scripts/`, `references/`,
655 `assets/`. The agent reads / runs these only when SKILL.md tells
656 it to. Use this tier for big artefacts, executables, rarely
657 needed detail. Cite each Tier-3 file from the body with a clear
658 "when to load / run it" trigger; uncited Tier-3 files are dead
659 weight.
660
661 ## 4. Writing the description (Tier 1, highest leverage)
662
663 The description's CORE PURPOSE is to TRIGGER skill loading -- not to
664 introduce or sell the skill. The agent's matcher reads it against the
665 incoming user prompt; if it doesn't match, none of your Tier-2 / Tier-3
666 work matters because the skill never fires.
667
668 ### Required ingredients (every description should contain ALL):
669
670 1. **Task object(s)** -- the concrete things the skill operates on.
671 Examples: `Dockerfile`, `Express middleware`, `JSON schema`,
672 `LaTeX table`, `pytest fixture`, `verifier output`, `npm package`,
673 `bearer token`, `trajectory.json`. List 2-4.
674 2. **Action verb(s)** -- what the skill DOES with those objects.
675Examples: `inspect`, `debug`, `validate`, `summarize`, `patch`,
676`compare`, `extract`, `aggregate`, `refactor`, `migrate`. List 2-4.
677 3. **WHEN scenarios** -- 2-3 concrete trigger contexts in
678 "Use this skill when..." form. Include INDIRECT phrasings users
679 actually type ("the test is timing out", "the verifier reports

```

```

680     X", "the JSON file in /tmp doesn't validate"). Synonyms count.
681 4. Optional: boundary conditions -- "Do not use this skill for
682 X / Y" lines. Reduces false-positive triggers when neighbouring
683 skills exist.
684
685 ### Style:
686
687 - Third-person / imperative voice: "This skill should be used
688 when..." or "Use when...". NEVER "I help with..." / "you can use
689 this to...".
690 - ≤1024 chars hard cap. Aim for 4-8 lines.
691 - Cover INDIRECT triggers (the user does not name the domain):
692 "the failing test mentions <error>", "the spreadsheet attached",
693 "the file in /tmp".
694
695 ### Banned patterns (REJECTED):
696
697 - Generic catch-all words as the only signal: `debug`, `fix`,
698 `code`, `bug`, `task`, `improve`, `helper`, `assist`. These match
699 everything = they match nothing.
700 - Meta-talk about the skill itself: `a structured workflow for...`,
701 `this skill helps with...`, `records past experience`,
702 `saves the agent from...`. Dilutes keyword signal.
703 - Self-referential phrasing: `This skill records what we learned
704 in trial X`. Skills describe TASK DISTRIBUTIONS and TRANSFERABLE
705 OPERATIONS, not their own provenance.
706 - Walking the body: don't put the workflow steps in the description.
707 Description = WHEN to fire; body = WHAT to do.
708
709 ### Concrete examples:
710
711 GOOD (auth-bypass family in Express/Node):
712 "Diagnose and fix bearer-token / JWT authentication-bypass bugs
713 in Node.js / Express middleware where invalid or malformed
714 tokens leak protected user data. Use when an auth middleware
715 (`tokenValidator`, `roleChecker`, `tokenPolicy`) defers errors
716 instead of returning 401, when `/api/users`-style routes return
717 user records under bad tokens, when soft-fail / monitor-only
718 auth modes mask 401-worthy failures, or when the verifier
719 reports user-data fields (`email`, `phone`, `ssn_last4`)
720 leaking through 200 responses. Do not use this skill for plain
721 role/RBAC bugs where authentication itself is correct."
722
723 POOR (does NOT trigger reliably):
724 "Use when debugging a software bug. Provides a structured,
725 step-by-step workflow for identifying the root cause."
726 -- generic; no task object, no domain keyword; matches every
727 coding task = matches none in practice.
728
729 ## 5. Body best practices
730
731 The body is what the agent reads AFTER the description triggers. Its
732 job is PROCEDURAL: tell the agent what to do, in order, with
733 explicit branches. It is NOT a paper, a textbook chapter, or a
734 narrative summary.
735
736 ### What the body MUST cover (when applicable):
737
738 - Goal statement -- one sentence: what does success look like.
739 - When to use -- echo the description's WHEN scenarios, more
740 concretely. May include "do not use when..." boundaries.
741 - Workflow -- numbered or `[ ]`-checklist of action steps in
742 order. Each step starts with an action verb ("First inspect...",
743 "Then verify...", "If X, do Y; else do Z."). NOT prose paragraphs.
744 - Gotchas -- concrete near-miss / failure patterns the agent
745 will hit otherwise (soft-deletes, ID renames, health checks that
746 lie, off-by-one timezone drift). Decision-rule form.
747 - Resource map -- a short table or list mapping triggers to
748 Tier-3 files: "Run `scripts/X.py` when ...", "Read
749 `references/Y.md` if ...", "Use `assets/Z.tmpl` to format ...".
750 REQUIRED whenever the skill has Tier-3 files.
751 - Output format / completion criteria -- when the task expects
752 a specific output shape, embed a literal template (agents
753 pattern-match against concrete structures better than prose).
754 - Uncertainty handling -- "if information X is missing, do Y
755 before proceeding" / "if validator output is ambiguous, default
756 to the SAFER branch". Tells the agent how to act under partial
757 information rather than guessing.
758
759 ### Style requirements:
760

```

```

761 - **Action-oriented language.** Every step starts with a verb in
762 imperative or "First / Then / If / Avoid" form. NO sentences like
763 "It is important to note that...".
764 - **Decision rules over advice.** "If <pattern>, do <action>" beats
765 "consider edge cases" / "be careful".
766 - **Defaults over menus.** Pick ONE recommended path; relegate
767 alternatives to a brief "Alternatives" line. Forcing the agent to
768 re-decide every call wastes tokens.
769 - **Match specificity to fragility.** PRESCRIPTIVE on fragile /
770 destructive / order-sensitive steps; PERMISSIVE on creative
771 steps. Mixing both within one SKILL.md is fine.
772 - **One worked example** beats exhaustive documentation. Trust the
773 agent's general competence on syntax / language basics.
774 - **Validation loop.** When the workflow is iterative: do-work,
775 run-validator (a `scripts/` file or shell command), fix, repeat.
776 State the validator command explicitly.
777 - **Plan-validate-execute** for batch / destructive ops: agent
778 produces a structured plan, validates against a source of truth,
779 THEN executes.
780
781 ### What the body MUST NOT contain:
782
783 - Background essays / theory dumps that the agent already knows
784 (basic Python syntax, "what is JWT", "what is Docker").
785 - A literal log of any single trial's trajectory.
786 - Hard-coded answers / fixes for one task instance ("change line
787 43 of tokenValidator.js to..." for a specific repo). Encode the
788 PATTERN ("when soft-fail logic defers auth errors, remove the
789 defer branch and return 401 + log") instead.
790 - Long enumerations (full enum tables, exhaustive API listings) --
791 move those to `references/<topic>.md`.
792 - Resources without a citation. If you bundle `scripts/foo.py` you
793 MUST tell the agent in the body: WHEN to run it, exact command,
794 expected output, how to interpret failure.
795
796 ### The body must be READABLE COLD by a future agent.
797
798 Assume the agent has zero memory of any past trial. Don't write
799 "as before" / "as in the previous task" / "the same fix as last
800 time". Every reference must be self-contained.
801
802 ## 6. Tier-3 directories
803
804 The agent's mental model: it RUNS files in `scripts/`, READS files in
805 `references/`, COPIES files in `assets/`. Place each file by its
806 intended interaction.
807
808 Every Tier-3 file MUST be cited from SKILL.md with a clear
809 "when to run / read / copy" trigger. Uncited Tier-3 files are dead
810 weight and rejected.
811
812 Naming (all three dirs):
813 - Specific verb_object pattern preferred:
814   `validate_skill_frontmatter.py`, `extract_failure_modes.py`,
815   `compare_skill_versions.py`, `failure_patterns.md`,
816   `result_schema.json`, `report_template.md`.
817 - REJECTED: vague names like `helper.py`, `notes.md`, `more.md`,
818   `stuff.txt`, `template2.md`, `old.json`, `doc1.md`.
819
820 ---
821
822 `scripts/` -- EXECUTABLE CODE the agent RUNS.
823
824 What goes here: deterministic, repeatable operations. Examples for
825 benchmarking-style domains: `parse_trajectory.py`,
826 `extract_failure_modes.py`, `validate_skill_frontmatter.py`,
827 `check_skill_patch.py`, `summarize_verifier_logs.py`,
828 `aggregate_results.py`, `compare_quotes.py`.
829
830 When to bundle one: trace shows the agent RE-DERIVING the same
831 logic, OR the step is fragile and a tested script beats freeform
832 generation.
833
834 Hard rules (REJECTED otherwise):
835 - **CLI**: clear arg interface; `--help` with brief description +
836   flags + examples.
837 - **Output**: structured (JSON / CSV / TSV) on stdout;
838   diagnostics on stderr; deterministic given same inputs.
839 - **Errors**: explain WHAT went wrong, WHAT was expected, WHAT to
840   try. NOT bare tracebacks.
841 - **Exception handling** for common failure modes: missing file,

```

```

842     malformed JSON / YAML, schema invalid, missing required field.
843 - **Idempotent** (agents retry).
844 - **Closed-set / enum** params for ambiguous choices.
845 - **--dry-run** for destructive ops; non-default for any
846   deletion / overwrite. NEVER default to deleting files,
847   overwriting results, or clearing dirs.
848 - **No interactive prompts**; agents run non-interactive shells.
849 - **No silent network installs**; declare deps inline (PEP 723
850   `# /// script` for Python; `npm:` for Deno; etc.) or in
851   SKILL.md `compatibility` field.
852 - **Meaningful exit codes**; documented in `--help`.
853 - **Predictable output size**; support `--output FILE` for large.
854
855 SKILL.md must specify, per script: WHEN to run, exact command,
856 expected output shape, how to interpret failure.
857
858 ---
859
860 `references/` -- ON-DEMAND DOCUMENTATION the agent READS.
861
862 What goes here: long-form material that would bloat SKILL.md if
863 inlined. Each file serves ONE clear topic. Examples for a
864 benchmarking / self-evolving domain:
865 - `failure_patterns.md` -- catalogue of TRANSFERABLE failure modes
866   (NOT single-trial details).
867 - `update_policy.md` -- when a skill update is allowed vs.
868   rejected.
869 - `procedural_skill_rubric.md` -- what makes a skill "good".
870 - `benchmark_task_schema.md` -- task spec / verifier interface.
871 - `trajectory_analysis_protocol.md` -- step-by-step trace review.
872 - `skill_patch_rubric.md` -- patch acceptance criteria.
873
874 Conventions:
875 - One topic per file. Filename describes the topic in 2-4 words.
876 - Long files (>200 lines): start with a TOC.
877 - SKILL.md links each reference DIRECTLY: "Read
878   `references/failure_patterns.md` if the verifier reports
879   pattern Y". NO daisy-chained jumps (ref → ref → ref).
880 - NO `references/all.md` dumping ground.
881 - NO obviously expired or task-instance-specific content unless
882   explicitly marked.
883
884 ---
885
886 `assets/` -- STATIC RESOURCES the agent COPIES / FILLS / EMBEDS
887 verbatim (never executed, rarely "read for understanding").
888
889 What goes here: document templates, config skeletons, prompt
890 scaffolds, lookup tables, JSON / YAML schemas, LaTeX templates,
891 sample input/output pairs. Examples:
892 - `skill_template.md`
893 - `skill_patch_template.md`
894 - `result_schema.json`
895 - `evaluation_table_template.tex`
896 - `report_template.md`
897
898 Conventions:
899 - Files are STRUCTURED, REUSABLE, READY to drop in with minimal
900   edits.
901 - SKILL.md tells the agent WHEN to use each: "Use
902   `assets/report_template.md` as the output structure; replace
903   `{placeholders}`".
904 - REJECTED here: long manuals (those go to `references/`),
905   backup / dead files, random notes, unused data.
906
907 ## 7. One-off commands (no `scripts/` needed)
908
909 If an existing tool already does the job, reference it directly with a
910 pinned version: `uvx ruff@0.8.0 check .`, `npx eslint@9 --fix .`, etc.
911 State runtime prerequisites in SKILL.md or in the `compatibility`
912 frontmatter field.
913
914 ## 8. Self-evolving skill discipline (HARD)
915
916 You are part of an evolving library: skills get revised after each
917 trial. That makes the line between "transferable knowledge" and
918 "trial-specific noise" critical. Discipline:
919
920 ### What a skill MUST be:
921
922 - A description of a TASK DISTRIBUTION + the TRANSFERABLE OPERATIONS

```

```

923     that solve it.
924 - A workflow that works on tasks the current trace did NOT cover.
925 - A library of failure patterns + decision rules that generalize.
926
927 ### What a skill MUST NOT be:
928
929 - A log of any single trajectory.
930 - A specific answer to one task instance ("the fix is to delete
931   line 43 of `tokenValidator.js`").
932 - A scrapbook of "what we learned in trial X".
933
934 ### Update policy (applies to every revise / propose call):
935
936 - **Update when** the trace shows a failure pattern that is likely
937   REPEATABLE across the family, AND the fix can be encoded as a
938   decision rule / workflow step / Tier-3 artefact that future agents
939   can apply WITHOUT having seen this trial.
940 - **Do NOT update** for a one-off quirk that only this task instance
941   exhibits. Silently accept the existing skill (NoOp-equivalent
942   revise: return SKILL.md verbatim).
943 - **Do NOT inflate** the body just to "have something to say".
944   If the trace teaches nothing new, NoOp.
945 - **NEVER write task-specific answers** into SKILL.md / references/
946   / assets/. That pollutes the library and biases future tasks.
947
948 ### Patch self-review checklist (run mentally before emitting JSON):
949
950 - [ ] Is every change driven by a pattern that will recur, not by
951   this trial's specific symptoms?
952 - [ ] Could a future agent, given only the new SKILL.md (no trace),
953   apply the workflow to a DIFFERENT task in this family?
954 - [ ] Is the description still a strong trigger ( $\geq 5$  domain
955   keywords; concrete WHEN scenarios)?
956 - [ ] Does every Tier-3 file have a clear "when to run / read /
957   copy" trigger in SKILL.md?
958 - [ ] Did I avoid copy-pasting trajectory text or task-instance
959   identifiers (file paths specific to this repo, exact line
960   numbers, the literal failing input)?
961
962 If any answer is "no", revise the patch before submitting.
963 =====
964 ""
965

```

966 Revision prompt templates

```

967 _REVISE_SYSTEM = ""\
968 You are a skill-evolution assistant maintaining a shared library of
969 procedural skills used by an LLM agent. Your role on this call is to
970 REVISE existing SKILL.md content (and optionally bundled Tier-3 files)
971 based on a recent task trial. Output STRICT JSON.
972
973 "" + _SKILL_MINDSET + _SPEC_BLOCK + ""
974
975
976 =====
977 HOW TO PROCESS A REVISION REQUEST
978 =====
979
980 The user message provides, in order:
981   ## Revision mode + mode-specific instruction
982   ## Trial outcome (PASS or FAIL banner + verifier breakdown + diagnosis_rule)
983   ## Compacted trajectory (the agent's recent run)
984   ## Target skills (slug list + task family + current SKILL.md content)
985
986 Apply these workflow rules (in addition to the spec block above):
987
988 1. **Apply the diagnosis_rule** provided in the trial-outcome section.
989   The PASS branch and the FAIL branch ask for different things; do NOT
990   substitute one for the other. If passed and the trajectory shows
991   nothing the skill can absorb, return upsert_files containing only the
992   existing SKILL.md text verbatim -- the parser treats that as NoOp.
993
994 2. **Respect the layering** (see Progressive Disclosure in the spec):
995   - Don't bloat SKILL.md with material that belongs in `references/`
996     (long enums, edge-case catalogues, vendored API docs).
997   - Don't shrink the `description` for token savings -- discoverability
998     beats a few chars.
999   - Don't change `name` unless operation_type=replace; renaming breaks
1000     the folder-slug match.
1001   - **`description` quality is the trigger surface** (Tier-1, always
1002     in future agents' context). When you revise, AUDIT the existing

```

```

1003 `description`. If it is generic ("Use when debugging a software
1004 bug.") or lacks domain keywords, REWRITE it to satisfy ALL of:
1005 a. <=1024 chars, imperative voice.
1006 b. AT LEAST 5 concrete keywords from the trajectory's domain
1007 (file types, tool names, error symptoms, library names,
1008 API verbs, domain terms). Generic words ("debug", "fix",
1009 "code", "bug", "task") DO NOT count.
1010 c. WHAT + WHEN: 2-3 concrete trigger contexts including
1011 INDIRECT ones (where the user does NOT name the skill
1012 domain).
1013 d. No meta-talk ("a structured workflow for...", "this skill
1014 helps with..."). It dilutes keyword signal.
1015 A weak description = the skill never fires on future tasks =
1016 all your other revisions are wasted.
1017
1018 3. **operation_type semantics**:
1019 - `revise` -- normal edit; same name, same intent.
1020 - `narrow` -- tighten `description` / `## When to use` so the skill no
1021 longer triggers on the failing context.
1022 - `replace` -- full rewrite; only when minor edits cannot fix the
1023 pattern.
1024 - `create` -- introduces a new skill (see rule 5; must be in the
1025 same family as the current task).
1026
1027 4. **Tier-3 files** (scripts/, references/, assets/) on EXISTING skills
1028 MUST be cited from SKILL.md with a clear "when to read / run / copy"
1029 trigger. Uncited Tier-3 files are dead weight -- the parser rejects
1030 them.
1031
1032 5. **Update existing vs create new (SAME FAMILY ONLY).**
1033
1034 When the family already has skills (shown in the "Skills in family"
1035 block of the user message), your DEFAULT action is to SELECT ONE OR
1036 MORE of them and REFINE them in place. Creating a brand-new sibling
1037 is the exception, not the default.
1038
1039 ADDITIVE BIAS for revisions:
1040 - PRESERVE the existing SKILL.md structure, naming, and intent
1041 unless the trajectory shows specific content is wrong, misleading,
1042 or unsafe.
1043 - PREFER ADDING over REPLACING: a new Gotcha, a tightened Workflow
1044 step, a refined `## When to use` clarifier, an extra cited Tier-3
1045 file, a sharper one-line summary in the frontmatter.
1046 - You MAY rewrite a section ONLY if the trajectory shows that
1047 section was wrong; cite the failing step in `summary`.
1048 - DO NOT shrink, gut, or wholesale-rewrite an existing SKILL.md
1049 "for clarity" / token savings -- accumulated knowledge IS the
1050 point of the library.
1051 - If the SKILL.md is fundamentally misaligned with the trajectory
1052 (rare), use operation_type=`replace` -- NEVER `create` with the
1053 same slug (see slug↔op pairing below).
1054
1055 SLUG ↔ operation_type pairing (HARD CONSTRAINT):
1056 - Reusing a slug already shown in "Skills in family" → op MUST be
1057 `revise` / `narrow` / `replace`. NEVER `create`.
1058 - `create` → MUST use a brand-new slug not visible in that block.
1059 - Violations are REJECTED by the parser.
1060
1061 You MAY create a new skill IFF the trajectory exposes a genuinely
1062 DISTINCT capability the existing skill is not the right place for
1063 (different workflow / problem space / prerequisites).
1064
1065 To create a new skill in the task's family:
1066 a. Pick a NEW slug. Naming rules (mandatory):
1067 - Lowercase ASCII letters, digits, and `` only (kebab-case)
1068 - Describes the capability concisely (3-6 hyphen-separated words)
1069 - Globally unique across ALL families. Check the existing slug
1070 list in the user message and avoid any visible name.
1071 - Do NOT include the family_id in the slug; the parser composes
1072 the formal id as `<task_family_id>.<your_slug>`.
1073 b. Add EXACTLY ONE entry to upsert_files:
1074   "<your_slug>/SKILL.md": "<full SKILL.md content>"
1075   The folder name and SKILL.md frontmatter `name` MUST match.
1076 c. NO Tier-3 files for new skills. The parser REJECTS Tier-3 paths
1077 under a new slug. Tier-3 is reserved for revisions to existing
1078 skills (rule 4).
1079 d. The new skill is recorded as `<task_family_id>.<your_slug>` --
1080 you cannot create a skill in another family from this task.
1081 e. In `summary`, state you are creating a new skill and briefly
1082 justify why no existing skill could absorb the evidence.
1083

```

```

1084     Updating an existing skill AND creating a new skill in the same patch
1085     is allowed -- list both under upsert_files. Set operation_type to
1086     "create" if any new slug is involved, otherwise "revise" / "narrow"
1087     / "replace" per rule 3.
1088
1089     ## Required JSON output
1090
1091     Return EXACTLY one JSON object with these keys:
1092
1093     ---
1094     {
1095       "summary": "<one short sentence describing the change>",
1096       "operation_type": "revise" | "narrow" | "replace" | "create",
1097       "upsert_files": {
1098         "<slug>/SKILL.md": "<full new SKILL.md content>",
1099         "<slug>/scripts/<file>": "<optional, only if SKILL.md cites it>",
1100         "<slug>/references/<file>": "<optional, only if SKILL.md cites it>",
1101         "<slug>/assets/<file>": "<optional, only if SKILL.md cites it>"
1102       },
1103       "delete_paths": []
1104     }
1105     ---
1106
1107     Always include FULL file content -- never diffs. Include only files you
1108     actually edited or created. No commentary outside the JSON.
1109     ""
1110
1111     _REVISE_USER_TEMPLATE = ""\
1112     ## Revision mode: {mode}
1113
1114     {mode_instruction}
1115
1116     ## Trial outcome
1117
1118     {outcome_section}
1119
1120     ## Cumulative trajectory across this family's learning trials
1121
1122     The trace below concatenates ALL prior learning-block trials in this
1123     family (T1, T2, ...) plus the trial that just finished. Each chunk is
1124     delimited by a `## Past trial:` or `## Current trial:` header. Reflect
1125     across the full episodic history -- patterns that appear repeatedly are
1126     stronger evidence than single-trial signals.
1127
1128     ---
1129     {trajectory_summary}
1130     ---
1131
1132     ## Skills in family `{family_id_for_this_task}`
1133
1134     This family has {n_existing} active skill(s): {existing_slug_list}.
1135
1136     **Default action when skills exist: SELECT ONE OR MORE of them and
1137     REFINE in place.** Apply the additive bias from system rule 5 --
1138     preserve existing structure and content; ADD what the trajectory
1139     teaches (gotchas, tightened steps, refined frontmatter summary, cited
1140     Tier-3 files). Do NOT rewrite or shrink existing SKILL.md content
1141     unless the trajectory proves a specific section is wrong.
1142
1143     The retriever suggested {slug_list} as the most relevant edit point
1144     ("← retriever suggested" marker), but you are NOT constrained to it
1145     -- you may:
1146
1147     - revise the suggested skill (additive refinement; default),
1148     - revise multiple skills in this family in one patch,
1149     - revise a DIFFERENT skill in this family that the trajectory shows
1150       is actually wrong / incomplete,
1151     - leave one skill untouched and edit another,
1152     - create a NEW sibling skill ONLY if the trajectory exposes a
1153       genuinely distinct capability the existing skills cannot absorb
1154       (see system prompt rule 5),
1155     - return all current SKILL.md content verbatim if the trajectory
1156       adds nothing new (NoUp-equivalent).
1157
1158     **REMINDER (system rule 5 hard constraint):** the slugs listed above
1159     ({existing_slug_list}) MUST NOT be reused under operation_type="create".
1160     For those slugs use `revise` / `narrow` / `replace`. `create` is only
1161     valid with a brand-new slug.
1162
1163     {target_blocks}

```


1166

"""

1167 **Experience-based induction prompt templates**

```

1168 _INDUCE_SYSTEM = """\
1169 You are a skill-evolution assistant maintaining a shared library of
1170 procedural skills used by an LLM agent. Your role on this call is to
1171 INDUCE a NEW skill from an LLM agent's recent task experience -- this
1172 is the FIRST version of this skill in the library. Output STRICT JSON.
1173
1174 """ + _SKILL_MINDSET + _SPEC_BLOCK + """
1175
1176 =====
1177 HOW TO PROCESS AN INDUCTION REQUEST
1178 =====
1179
1180 The user message provides:
1181 ## Family id, latent skill id, slug
1182 ## Last task verdict (verifier_passed + failure_summary)
1183 ## Compacted trajectory (your only ground-truth evidence)
1184
1185 Apply these induction rules (in addition to the spec block above):
1186
1187 1. **The trajectory is your evidence.** Treat it like a debrief:
1188 - Where did the agent stumble, improvise, retry, or miss the obvious?
1189   Those moments become Gotchas, Workflow steps, or bundled scripts.
1190 - What context (file paths, schema names, exact APIs, conventions)
1191   did the agent have to discover at runtime? Encode it directly so
1192   future runs do not.
1193 - Skip what the agent already knows (language syntax, file formats,
1194   standard libraries). Only project- and domain-specific knowledge
1195   that the agent had to RE-DERIVE.
1196
1197 2. **Frontmatter** -- `name` and `description` (HARD constraints):
1198
1199   `name` MUST equal the slug given in the user message.
1200
1201   `description` is the entire trigger surface (Tier-1, always in
1202   future agents' context). A weak description = the skill never
1203   fires. REQUIREMENTS:
1204
1205   a. <=1024 chars, imperative voice ("Use this skill when...").
1206   b. **Keyword density**: list AT LEAST 5 concrete keywords
1207      extracted from the trajectory's task domain -- file types,
1208      tool names, error symptoms, library/framework names, API
1209      verbs, domain terms. Generic words ("debug", "fix", "code",
1210      "bug", "task") DO NOT count as keywords.
1211   c. **State WHAT (capability) AND WHEN (trigger contexts)**, not
1212      just one. The "when" should list 2-3 concrete trigger contexts
1213      -- including INDIRECT ones where the user does NOT name the
1214      skill domain ("the spreadsheet attached", "the file in /tmp",
1215      "the failing test mentions <error>"). Synonyms count.
1216   d. NO meta-talk about the skill itself ("a structured workflow
1217      for...", "this skill helps with...") -- agents skim
1218      descriptions and meta-talk dilutes keyword signal.
1219
1220 GOOD example (for a hypothetical auth-bypass family):
1221 "Diagnose and fix bearer-token / JWT authentication bypass
1222 bugs in Node.js / Express middleware where invalid tokens leak
1223 protected user data. Use when an Express auth middleware
1224 (tokenValidator, roleChecker, tokenPolicy) defers errors,
1225 when /api/users-style routes return user records under bad
1226 tokens, when soft-fail / monitor-only auth modes mask
1227 401-worthy failures, or when the verifier reports user-data
1228 fields (email, phone, ssn_last4) leaking through 200
1229 responses."
1230
1231 POOR example (does NOT trigger reliably):
1232 "Use when debugging a software bug. Provides a structured,
1233 step-by-step workflow for identifying the root cause."
1234 -- no domain keywords; "debug a software bug" matches
1235 everything = matches nothing.
1236
1237 3. **Recommended body sections** (omit ones you don't need):
1238 `# <title>` -> `## When to use` -> `## Workflow` -> `## Examples`
1239 -> `## Gotchas` -> `## Output template`.
1240 Gotchas is usually the single highest-value section -- write it
1241 when the trajectory shows any near-miss or recovery.
1242
1243 4. **Pick ONE default path** in the workflow; relegate alternatives to

```

```

1245     a brief "Alternatives" line.
1246
1247 5. **Tier-3 files** are optional. Add a `<slug>/scripts/<file>` only
1248 when the trajectory shows the agent re-deriving the same logic or
1249 the step is fragile enough that a tested script beats freeform
1250 generation. Use `<slug>/references/<file>` for long material loaded
1251 on demand. Use `<slug>/assets/<file>` for verbatim templates. Every
1252 Tier-3 file MUST be cited from SKILL.md body with a clear trigger;
1253 uncited Tier-3 files are dead weight.
1254
1255 ## Required JSON output
1256
1257 ---
1258 {
1259   "summary": "<one short sentence>",
1260   "operation_type": "create",
1261   "upsert_files": {
1262     "<slug>/SKILL.md": "<full SKILL.md, YAML frontmatter first>",
1263     "<slug>/scripts/<file>": "<optional, only if SKILL.md cites it>",
1264     "<slug>/references/<file>": "<optional, only if SKILL.md cites it>",
1265     "<slug>/assets/<file>": "<optional, only if SKILL.md cites it>"
1266   }
1267 }
1268 ---
1269
1270 Minimal acceptable output is just the `<slug>/SKILL.md` entry.
1271 """
1272
1273 _INDUCE_USER_TEMPLATE = """\
1274 ## Family id
1275
1276 {family_id}
1277
1278 ## Latent skill id (formal, internal handle)
1279
1280 {latent_skill_id}
1281
1282 ## Slug (folder name AND frontmatter `name` -- MUST be identical)
1283
1284 {slug}
1285
1286 ## Last task verdict
1287
1288 verifier_passed: {verifier_passed}
1289 failure_summary: {failure_summary}
1290
1291 ## Compacted trajectory (your only ground-truth evidence)
1292
1293 ---
1294 {trajectory_summary}
1295 ---
1296 """
1297

```

1299 Zero-shot generation prompt templates

```

1300 _ZERO_SHOT_SYSTEM = """\
1301 You are a skill-evolution assistant maintaining a shared library of
1302 procedural skills used by an LLM agent. Your role on this call is to
1303 DRAFT a procedural skill BEFORE the agent has executed any task in
1304 this family -- you only have the family label and a brief description,
1305 NO execution trace yet. Output STRICT JSON.
1306
1307 (Mindset note for ZERO-SHOT: cast a WIDE net on triggers and workflow
1308 shape, but do NOT invent project-specific gotchas / examples / numbers
1309 -- you have no evidence yet. T1's induction will fill those in once a
1310 real trace exists. The "comprehensive + adversarial" mindset below
1311 applies to the GENERAL category, not to invented specifics.)
1312
1313 """ + _SKILL_MINDSET + _SPEC_BLOCK + """
1314
1315 =====
1316 HOW TO PROCESS A ZERO-SHOT DRAFT REQUEST
1317 =====
1318
1319 The user message provides:
1320   ## Family id, latent skill id, slug
1321   ## Family name + description (your only inputs -- no trace, no outcome)
1322
1323 Apply these zero-shot rules (in addition to the spec block above):
1324

```

```

1325
1326 You are writing Tier 1 + Tier 2 ONLY. Tier 3 (`scripts/`, `references/`,
1327 `assets/`) MUST NOT be created yet -- wait until T1's real execution
1328 reveals what's worth caching.
1329
1330 1. **No invented gotchas, no invented examples.** You have no execution
1331 evidence. Inventing project quirks would mislead the agent. Leave
1332 `## Gotchas` and `## Examples` for T1 induction to fill in from the
1333 real trace.
1334
1335 2. **Frontmatter** -- `name` and `description` (HARD constraints):
1336
1337     `name` MUST equal the slug given in the user message.
1338
1339     `description` is the entire trigger surface (Tier-1, always in
1340     future agents' context). REQUIREMENTS:
1341
1342     a. <=1024 chars, imperative voice ("Use this skill when...").
1343     b. **Keyword density**: extract AT LEAST 5 concrete keywords
1344        from the FAMILY description / family name provided in the
1345        user message -- domain terms, tool names, error patterns,
1346        file types, library names. Generic words ("debug", "fix",
1347        "code", "bug") DO NOT count.
1348     c. **State WHAT (capability) AND WHEN (trigger contexts)**.
1349        Cast WIDE here (T1 induction will narrow once a real trace
1350        exists). List 3-5 trigger contexts including INDIRECT ones
1351        where the user does NOT name the skill domain.
1352     d. NO meta-talk about the skill itself ("a structured workflow
1353        for...", "this skill helps with...") -- dilutes signal.
1354
1355     POOR example (does NOT trigger reliably):
1356     "Use when debugging a software bug. Provides a structured,
1357     step-by-step workflow for identifying the root cause."
1358     -- generic, no domain keywords; matches everything = matches
1359     nothing.
1360
1361 3. **Body** (only the sections you can honestly write without a trace):
1362     - `# <title>` -- one line.
1363     - `## When to use` -- bullet list of trigger conditions (cast wider
1364       than the description).
1365     - `## Workflow` -- best-guess numbered procedure based on the family
1366       description. Prescriptive on obviously fragile / order-sensitive
1367       steps; permissive on creative ones.
1368     - `## Output template` -- only if the family description implies a
1369       fixed output shape; otherwise omit.
1370
1371 4. **One default path, no menus.** If multiple approaches are plausible,
1372     pick the most common one and move on.
1373
1374 5. **Keep the body under ~200 lines.** Post-T1 revision will refine it;
1375     over-investing now will mostly be rewritten.
1376
1377 ## Required JSON output
1378
1379 ---
1380 {
1381   "summary": "<one short sentence>",
1382   "operation_type": "create",
1383   "upsert_files": {
1384     "<slug>/SKILL.md": "<full SKILL.md content, YAML frontmatter first>"
1385   }
1386 }
1387 ---
1388
1389 Legacy single-key form is also accepted for backward compatibility:
1390 `{"summary": "...", "skill_md": "<SKILL.md content>"}`.
1391 """"
1392
1393
1394 _ZERO_SHOT_USER_TEMPLATE = """"\
1395 ## Family id
1396
1397 {family_id}
1398
1399 ## Latent skill id (formal, internal handle)
1400
1401 {latent_skill_id}
1402
1403 ## Slug (folder name AND frontmatter `name` -- MUST be identical)
1404
1405 {slug}

```

```

1406
1407 ## Family name
1408
1409 {name}
1410
1411 ## Family description
1412
1413 {description}
1414
1415 ## Output format
1416
1417 Return your response as a single JSON object exactly per the schema in the system prompt.
1418 """

```

1420 D Supplementary Experimental Results

1421 This appendix provides additional analyses that complement the main aggregate results.

1422 D.1 Exploring Whether Larger Skill Libraries Improve Frozen Evaluation

1423 This ablation asks whether agents can extract enough reusable information from their execution traces
1424 to enrich the skill library beyond a single SKILL.md instruction file. The trace contains concrete
1425 evidence: commands that were repeatedly re-derived, validation scripts the agent wished it had,
1426 schema details discovered at runtime, lookup tables, examples, and templates. A capable Skill Author
1427 should be able to turn such evidence into persistent resources under `scripts/`, `references/`, and
1428 `assets/`. The central question is therefore not simply whether larger libraries are better. It is whether
1429 trace evidence contains enough reusable structure for the authoring model to extract, package, and
1430 attach it to the right procedural trigger.

1431 We use Tier-3 forcing as a diagnostic for this question. If the author can extract useful information
1432 from traces, then ALWAYS+TIER3 should both produce richer libraries and preserve or improve
1433 frozen deployment success. If the author only produces additional files without reliable procedural
1434 value, then the library will grow while ESR remains flat or drops. The analysis below therefore
1435 separates two outcomes: *enrichment capacity*, measured by the number of persisted skill resources,
1436 and *useful enrichment*, measured by whether those resources improve ESR.

1437 D.1.1 Tier-3 Resource Bundling Prompt

1438 Skills in SKILLEVOLBENCH are intended to be reusable procedural memory rather than short textual
1439 hints. Intuitively, a richer skill should be more useful: executable helpers can remove brittle repeated
1440 commands, reference files can preserve long tables or API notes without bloating SKILL.md, and
1441 assets can provide reusable templates or schemas. The Tier-3 setting tests this intuition directly.
1442 It keeps the surrounding Skill Author interface fixed, but changes the revision instruction so that
1443 resource bundling is mandatory rather than optional.

1444 Importantly, the non-Tier-3 condition is not a ban on auxiliary files. The default `free_form` strategy
1445 allows the Skill Author to add `scripts/`, `references/`, or `assets/` when they materially help.
1446 By contrast, the `tier3_required` strategy forces each eligible revision to include at least one such
1447 resource under an existing skill. This creates a controlled comparison between ordinary free-form
1448 skill editing and an aggressive resource-bundling policy.

Shared structured edit interface

Return EXACTLY one JSON object with these keys:

```

{
  "summary": "<one short sentence describing the change>",
  "operation_type": "revise" | "narrow" | "replace" | "create",
  "upsert_files": {
    "<slug>/SKILL.md": "<full new SKILL.md content>",
    "<slug>/scripts/<file>": "<optional, only if SKILL.md cites it>",
    "<slug>/references/<file>": "<optional, only if SKILL.md cites it>",
    "<slug>/assets/<file>": "<optional, only if SKILL.md cites it>"
  }
}

```

1449

```
}  
}
```

Always include FULL file content -- never diffs. Include only files you actually edited or created. No commentary outside the JSON.

Without forced Tier-3: free-form revision mode

Make whatever edit best closes the gap exposed by the failure.
Edit SKILL.md and -- when it materially helps -- add executable
`scripts/`, on-demand `references/`, or `assets/` files.
Cite every Tier-3 file you add from the SKILL.md body with a
clear "when to read / run" trigger; uncited files will be rejected.
Do NOT inflate the body for its own sake; size the change to the gap.

With forced Tier-3: tier3-required revision mode

MANDATORY for this revision: upsert_files MUST include at least ONE new or updated file under `

For folder choice + naming + hard rules, follow system spec section 6.
The summary below is just a decision guide for picking the folder;
spec 6 is the binding contract for HOW to write the file.

DECISION GUIDE for this trial:

- Trace shows the agent re-deriving fragile multi-step logic, OR running the same ad-hoc command repeatedly
-> bundle as `and cite the exact command from SKILL.md.
- Trace shows the agent looking up long enum / API error table / vendored doc that would bloat SKILL.md
-> move to `("read this if X happens") in SKILL.md.
- Trace shows the agent reproducing a fixed output structure
(template / schema / config skeleton)
-> bundle as `

If a suitable Tier-3 file already exists in the target skill, UPDATE it in place -- do NOT create a near-duplicate.

ADDITIVE BIAS still applies: cite this Tier-3 file from SKILL.md with a clear "when to run / read / copy" trigger. Uncited files are dead weight and rejected.

The parser enforces the distinction above. In tier3_required mode, upsert_files must contain at least one Tier-3 path under an existing target skill. A SKILL.md-only revision is invalid for this setting. Thus, the comparison asks whether forcing richer skill artifacts improves reuse beyond ordinary free-form skill revision, while keeping the trajectory evidence, verifier feedback, and Skill Author call surface unchanged.

D.1.2 Results and Library Size

The results separate artifact extraction from functional enrichment. Forced Tier-3 authoring does make agents write richer libraries: the ALWAYS+TIER3 variants generally add more files under scripts/, references/, and assets/. This indicates that agents can identify trace-derived material that appears reusable. However, Table 9 and Fig. 5 show that this extraction step is not enough to guarantee better frozen deployment. Many points in Fig. 5 move rightward without moving upward: the library grows, but ESR stays flat or decreases.

The positive cases show that trace enrichment can be useful when the added resources capture stable procedures that the task-solving agent can actually reuse. CLAUDE OPUS 4.6 improves under SELFGEN-ALWAYS+TIER3, from 37.8% to 40.0% ESR, and GEMINI 3.1 PRO improves under

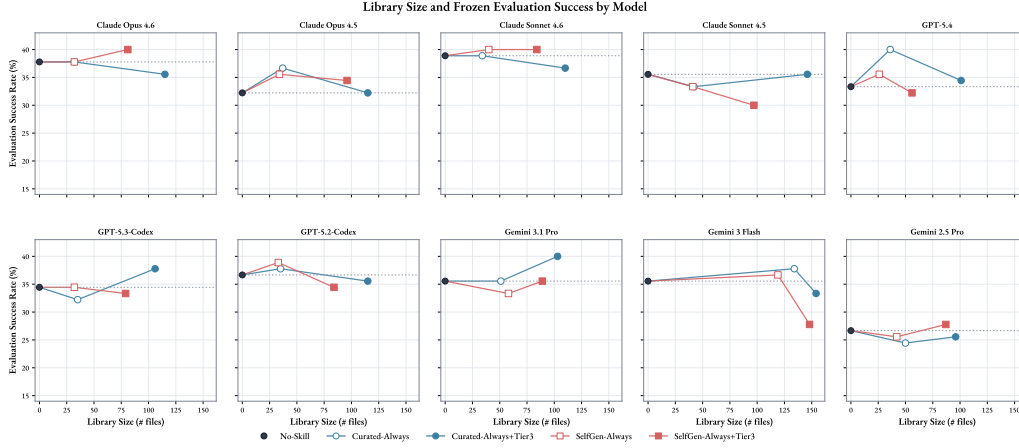


Figure 5: **Library size versus frozen evaluation success.** Each panel corresponds to one model. The horizontal axis is the final library size, measured as the number of `SKILL.md`, `references/`, `scripts/`, and `assets/` files. The vertical axis reports ESR. Open markers denote ordinary ALWAYS revision; filled markers denote ALWAYS+TIER3. Lines connect each memory condition to the same model’s NO-SKILL run.

1468 CURATED-ALWAYS+TIER3, from 35.6% to 40.0%. These results suggest that some traces do contain
 1469 procedural details that are not fully preserved in the compact `SKILL.md` body alone. In those cases,
 1470 Tier-3 files can serve their intended role: preserving reusable validation routines, reference material,
 1471 templates, or executable helpers that future agents can load on demand.

1472 The failure cases show where current trace enrichment breaks down. GPT-5.4 reaches its high-
 1473 est ESR with CURATED-ALWAYS, while the larger CURATED-ALWAYS+TIER3 and SELFGEN-
 1474 ALWAYS+TIER3 libraries reduce ESR. GEMINI 3 FLASH is the clearest overload case: its SELFGEN-
 1475 ALWAYS+TIER3 library is among the largest, but ESR drops from 35.6% under NO-SKILL to 27.8%.
 1476 CLAUDE SONNET 4.5 also drops from 35.6% under SELFGEN-ALWAYS+TIER3. These
 1477 drops suggest that forced enrichment can preserve trace details that look useful locally but do not
 1478 transfer. Added resources may encode episode-specific assumptions, over-specific validators, stale
 1479 context, or weakly triggered files that increase retrieval burden during frozen evaluation.

1480 Thus, the answer to the trace-enrichment question is only partially positive. Agents can extract more
 1481 information from traces and package it as persistent skill resources, but they do not yet reliably extract
 1482 the *right* information. The limiting factor is selective enrichment: deciding which trace details are
 1483 stable enough to persist, how to organize them as procedural resources, and when a future agent
 1484 should load them. Tier-3 therefore exposes the same abstraction bottleneck from another angle. More
 1485 capacity helps only when it is controlled; otherwise, richer libraries become procedural clutter rather
 1486 than more reliable skills.

1487 D.2 Environment-Level Success Patterns

1488 Fig. 6 shows that environment structure is another large source of variation. Across baselines, the gap
 1489 between the easiest and hardest environments is 67.3 percentage points for LSR and 42.1 points for
 1490 ESR. By contrast, the corresponding ranges across baselines are only 1.9 and 5.4 points. This suggests
 1491 that SKILLEVBENCH is not merely measuring a single global effect of adding memory. Instead,
 1492 it exposes domain-specific differences in how easily experience can be converted into reusable
 1493 procedural knowledge.

1494 The deployment roles reveal distinct bottlenecks. E2 Tool/API has the highest context-shift success,
 1495 with mean CSSR of 84.7%, indicating that agents can often reuse procedural patterns for changed API
 1496 contexts once the relevant interface discipline is available. E3 Data has the highest adversarial success,
 1497 with mean ARSR of 69.8%, but its mean CompSR is only 4.5%. Thus, robustness to adversarial
 1498 perturbations and skill composition are not interchangeable deployment capabilities. Similarly, E1
 1499 Code and E4 Docs are comparatively strong on composition, while E6 Communication remains

Table 9: **NO-SKILL vs. always-update Tier-3 ablations.** Values are success rates (%). Deltas report percentage-point differences from the corresponding NO-SKILL result. For RSR, the reference is NO-SKILL LSR because NO-SKILL has no replay phase. Red/blue/gray denote positive/negative/zero deltas.

Agent Harness	Model	Condition	LSR		RSR		ESR		CSSR		ARSR		CompSR	
			%	Δ	%	Δ	%	Δ	%	Δ	%	Δ	%	Δ
CLAUDE CODE	Claude Opus 4.6	NO-SKILL	38.9	—	—	—	37.8	—	46.7	—	36.7	—	30.0	—
		CURATED ALWAYS	42.2	+3.3	46.7	+7.8	37.8	0.0	40.0	-6.7	46.7	+10.0	26.7	-3.3
		CURATED ALWAYS+TIER3	45.6	+6.7	42.2	+3.3	35.6	-2.2	43.3	-3.3	40.0	+3.3	23.3	-6.7
		SELFGEN ALWAYS	42.2	+3.3	45.6	+6.7	37.8	0.0	43.3	-3.3	46.7	+10.0	23.3	-6.7
		SELFGEN ALWAYS+TIER3	44.4	+5.6	51.1	+12.2	40.0	+2.2	46.7	0.0	43.3	+6.7	30.0	0.0
	Claude Opus 4.5	NO-SKILL	42.2	—	—	—	32.2	—	40.0	—	40.0	—	16.7	—
		CURATED ALWAYS	42.2	0.0	44.4	+2.2	36.7	+4.4	43.3	+3.3	43.3	+3.3	23.3	+6.7
		CURATED ALWAYS+TIER3	45.6	+3.3	45.6	+3.3	32.2	0.0	33.3	-6.7	36.7	-3.3	26.7	+10.0
		SELFGEN ALWAYS	41.1	-1.1	40.0	-2.2	35.6	+3.3	40.0	0.0	40.0	0.0	26.7	+10.0
		SELFGEN ALWAYS+TIER3	38.9	-3.3	38.9	-3.3	34.4	+2.2	46.7	+6.7	40.0	0.0	16.7	0.0
	Claude Sonnet 4.6	NO-SKILL	37.8	—	—	—	38.9	—	40.0	—	50.0	—	26.7	—
		CURATED ALWAYS	40.0	+2.2	44.4	+6.7	38.9	0.0	46.7	+6.7	43.3	-6.7	26.7	0.0
		CURATED ALWAYS+TIER3	41.1	+3.3	44.4	+6.7	36.7	-2.2	40.0	0.0	43.3	0.0	26.7	0.0
		SELFGEN ALWAYS	41.1	+3.3	46.7	+8.9	40.0	+1.1	40.0	0.0	50.0	0.0	30.0	+3.3
		SELFGEN ALWAYS+TIER3	42.2	+4.4	44.4	+6.7	40.0	+1.1	43.3	+3.3	46.7	-3.3	30.0	+3.3
	Claude Sonnet 4.5	NO-SKILL	41.1	—	—	—	35.6	—	40.0	—	46.7	—	20.0	—
		CURATED ALWAYS	37.8	-3.3	38.9	-2.2	33.3	-2.2	40.0	0.0	40.0	-6.7	20.0	0.0
		CURATED ALWAYS+TIER3	40.0	-1.1	44.4	+3.3	35.6	0.0	43.3	+3.3	40.0	-6.7	23.3	+3.3
		SELFGEN ALWAYS	38.9	-2.2	44.4	+3.3	33.3	-2.2	40.0	0.0	40.0	-6.7	20.0	0.0
		SELFGEN ALWAYS+TIER3	40.0	-1.1	36.7	-4.4	30.0	-5.6	33.3	-6.7	33.3	-13.3	23.3	+3.3
CODEX CLI	GPT-5.4	NO-SKILL	43.3	—	—	—	33.3	—	43.3	—	33.3	—	23.3	—
		CURATED ALWAYS	43.3	0.0	42.2	-1.1	40.0	+6.7	40.0	-3.3	50.0	+16.7	30.0	+6.7
		CURATED ALWAYS+TIER3	44.4	+1.1	40.0	-3.3	34.4	+1.1	46.7	+3.3	33.3	0.0	23.3	0.0
		SELFGEN ALWAYS	44.4	+1.1	43.3	0.0	35.6	+2.2	46.7	+3.3	40.0	+6.7	20.0	-3.3
		SELFGEN ALWAYS+TIER3	44.4	+1.1	46.7	+3.3	32.2	-1.1	40.0	-3.3	36.7	+3.3	20.0	-3.3
	GPT-5.3-Codex	NO-SKILL	44.4	—	—	—	34.4	—	36.7	—	46.7	—	20.0	—
		CURATED ALWAYS	42.2	-2.2	45.6	+1.1	32.2	-2.2	40.0	+3.3	40.0	-6.7	16.7	-3.3
		CURATED ALWAYS+TIER3	42.2	-2.2	47.8	+3.3	37.8	+3.3	46.7	+10.0	43.3	-3.3	23.3	+3.3
		SELFGEN ALWAYS	44.4	0.0	48.9	+4.4	34.4	0.0	36.7	0.0	43.3	-3.3	23.3	+3.3
		SELFGEN ALWAYS+TIER3	46.7	+2.2	50.0	+5.6	33.3	-1.1	36.7	0.0	43.3	-3.3	20.0	0.0
	GPT-5.2-Codex	NO-SKILL	43.3	—	—	—	36.7	—	40.0	—	53.3	—	16.7	—
		CURATED ALWAYS	44.4	+1.1	45.6	+2.2	37.8	+1.1	46.7	+6.7	43.3	-10.0	23.3	+6.7
		CURATED ALWAYS+TIER3	46.7	+3.3	45.6	+2.2	35.6	-1.1	46.7	+6.7	43.3	-10.0	16.7	0.0
		SELFGEN ALWAYS	46.7	+3.3	46.7	+3.3	38.9	+2.2	50.0	+10.0	43.3	-10.0	23.3	+6.7
		SELFGEN ALWAYS+TIER3	46.7	+3.3	44.4	+1.1	34.4	-2.2	40.0	0.0	40.0	-13.3	23.3	+6.7
GEMINI CLI	Gemini 3.1 Pro	NO-SKILL	40.0	—	—	—	35.6	—	40.0	—	46.7	—	20.0	—
		CURATED ALWAYS	40.0	0.0	45.6	+5.6	35.6	0.0	40.0	0.0	40.0	-6.7	26.7	+6.7
		CURATED ALWAYS+TIER3	45.6	+5.6	41.1	+1.1	40.0	+4.4	43.3	+3.3	50.0	+3.3	26.7	+6.7
		SELFGEN ALWAYS	42.2	+2.2	41.1	+1.1	33.3	-2.2	33.3	-6.7	43.3	-3.3	23.3	+3.3
		SELFGEN ALWAYS+TIER3	45.6	+5.6	45.6	+5.6	35.6	0.0	40.0	0.0	40.0	-6.7	26.7	+6.7
	Gemini 3 Flash	NO-SKILL	40.0	—	—	—	35.6	—	30.0	—	53.3	—	23.3	—
		CURATED ALWAYS	43.3	+3.3	37.8	-2.2	37.8	+2.2	46.7	+16.7	36.7	-16.7	30.0	+6.7
		CURATED ALWAYS+TIER3	38.9	-1.1	41.1	+1.1	33.3	-2.2	36.7	+6.7	43.3	-10.0	20.0	-3.3
		SELFGEN ALWAYS	35.6	-4.4	40.0	0.0	36.7	+1.1	36.7	+6.7	43.3	-10.0	30.0	+6.7
		SELFGEN ALWAYS+TIER3	43.3	+3.3	40.0	0.0	27.8	-7.8	33.3	+3.3	36.7	-16.7	13.3	-10.0
	Gemini 2.5 Pro	NO-SKILL	30.0	—	—	—	26.7	—	26.7	—	30.0	—	23.3	—
		CURATED ALWAYS	31.1	+1.1	26.7	-3.3	24.4	-2.2	26.7	0.0	23.3	-6.7	23.3	0.0
		CURATED ALWAYS+TIER3	30.0	0.0	27.8	-2.2	25.6	-1.1	33.3	+6.7	23.3	-6.7	20.0	-3.3
		SELFGEN ALWAYS	31.1	+1.1	34.4	+4.4	25.6	-1.1	40.0	+13.3	20.0	-10.0	16.7	-6.7
		SELFGEN ALWAYS+TIER3	23.3	-6.7	21.1	-8.9	27.8	+1.1	36.7	+10.0	26.7	-3.3	20.0	-3.3

difficult across all conditions. In particular, E6 obtains 0.0% CompSR for every baseline, highlighting communication and scheduling as a hard setting for multi-constraint procedural reuse.

The heatmaps also show that skill-based abstraction does not uniformly dominate direct episodic reuse. RAW-TRAJECTORY is the strongest baseline by mean RSR, ESR, ARSR, and CompSR, with averages of 48.2%, 37.6%, 44.7%, and 25.7%, respectively. It is especially competitive in E4 Document Parsing & Transformation and E5 Research & Information Synthesis, where retaining concrete traces can preserve useful task-specific details that may be lost during abstraction. At the same time, replay improvements in structured environments such as E1 Code and E4 Docs indicate

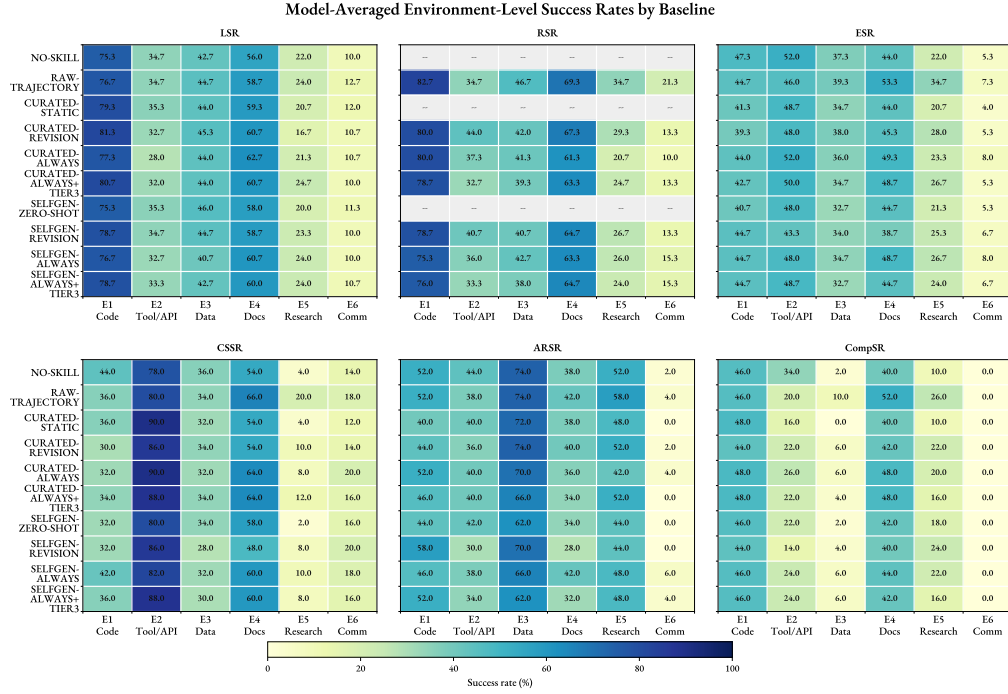


Figure 6: **Environment-level success rates by baseline.** Each cell reports the success rate averaged over model-harness runs for a fixed baseline and environment. Columns use short environment names: E1 Code Debugging & Modification, E2 Tool & API Orchestration, E3 Data Processing & Structured Query, E4 Document Parsing & Transformation, E5 Research & Information Synthesis, and E6 Communication & Scheduling. LSR and RSR measure acquisition and replay success, respectively. ESR measures frozen deployment success, while CSSR, ARSR, and CompSR break deployment into context-shift, adversarial, and composition roles. Gray cells indicate variants without replay.

that experience can become reusable after acquisition. Together, these patterns suggest that skill evolution is most reliable when the environment supports stable procedural abstractions, whereas open-ended information and communication tasks remain sensitive to missing context, underspecified constraints, and brittle composition.

D.3 Cost-Success Analysis

Overall Pattern. Fig. 7 shows that memory mechanisms do not form a uniform cost-success frontier across models. Most variants increase cost per attempted task relative to NO-SKILL, but the corresponding ESR gain is highly model- and variant-dependent. This is visible from the direction of the paired segments: many move rightward with little vertical movement, while only a smaller subset move both rightward and upward.

Static Curated Skills Are Not Sufficient. CURATED-STATIC underperforms NO-SKILL on average. It improves 2 models, ties 1, and hurts 7, with a mean ESR change of -2.44 percentage points. This suggests that fixed curated procedures are not automatically useful under deployment shift. When the skill cannot be revised, it may overfit to the canonical procedure or introduce misleading constraints for later roles. The cost also increases by $+\$0.077$ per attempted task on average, so the static skill condition is generally dominated by NO-SKILL and RAW-TRAJECTORY.

Revision Helps Curated Skills, but Only Partially. Allowing curated skills to be revised makes the curated family more competitive, but the effect is still mixed. CURATED-REVISION has an average change of -0.67 percentage points, with 3 wins, 4 ties, and 3 losses. CURATED-ALWAYS is the strongest curated variant, with a mean gain of $+0.78$ percentage points and 4 wins, 3 ties, and 3 losses. The benefit is concentrated in particular models: GPT-5.4 improves by $+6.7$ percentage points under CURATED-ALWAYS, and OPUS 4.5 improves by $+4.4$ percentage points. Thus, frequent curated-skill updating can help, but it does not produce a model-agnostic advantage.

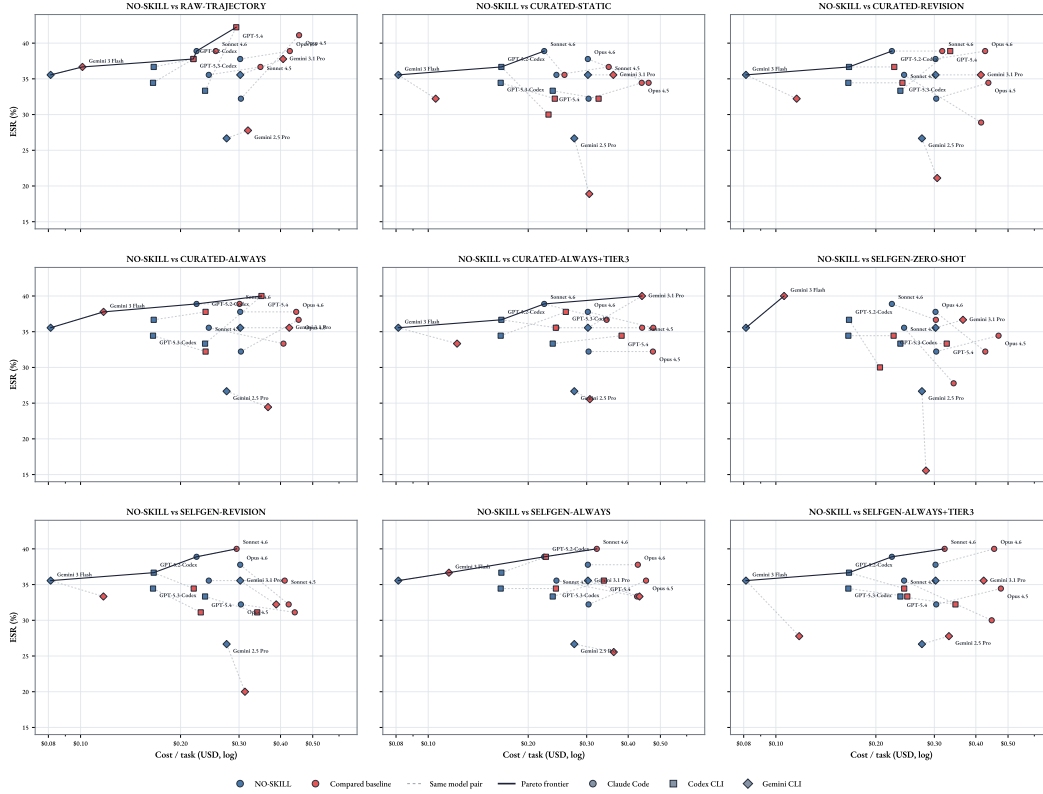


Figure 7: **Model-specific cost–success trade-offs relative to NO-SKILL.** Each panel compares NO-SKILL against one memory variant using the same ten models. The x -axis reports cost per attempted task in USD on a log scale, and the y -axis reports frozen evaluation success rate (ESR). Blue markers denote the NO-SKILL run and red markers denote the compared variant. Dashed gray segments connect the two points from the same model. Marker shape indicates the agent harness. The dark line marks the Pareto frontier within each pairwise comparison.

1531 **Tier-3 Authoring Does Not Reliably Improve the Frontier.** CURATED-TIER3 and SELFGEN-
1532 TIER3 do not produce stable gains despite their higher authoring cost. CURATED-TIER3 has a mean
1533 ESR change of 0.00 percentage points, with 3 wins, 2 ties, and 5 losses, while increasing cost by
1534 +\$0.119 per attempted task on average. SELFGEN-TIER3 averages -1.11 percentage points, with
1535 4 wins, 1 tie, and 5 losses. These results suggest that a more detailed or more strongly constrained
1536 authoring prompt is not automatically better. In this setting, additional authoring effort can increase
1537 cost without reliably improving deployment success.

1538 **Self-Generated Skills Need Frequent Updates.** The self-generated variants show a clear distinc-
1539 tion between update policies. SELFGEN-REVISION, which updates only after eligible failures, is
1540 weak: it improves only 1 model, ties 1, and hurts 8, with an average change of -2.56 percentage
1541 points. SELFGEN-ALWAYS is substantially healthier, improving 5 models, tying 2, and hurting 3, with
1542 an average change of $+0.44$ percentage points. This supports the interpretation that self-generated
1543 procedural memory benefits from dense update opportunities. Failure-only revision appears too
1544 sparse or too reactive to reliably produce reusable skills.

1545 **Zero-Shot Skill Generation Is Brittle.** SELFGEN-ZEROSHOT is also negative on average, with a
1546 mean ESR change of -2.56 percentage points. It helps GEMINI 3 FLASH by $+4.4$ percentage points,
1547 but hurts GEMINI 2.5 PRO by -11.1 percentage points. The large spread indicates that metadata-only
1548 skills can act as useful prior knowledge for some models but harmful procedural bias for others. This
1549 brittleness motivates separating zero-shot initialization from experience-based skill evolution in the
1550 main analysis.

1551 **Model Dependence.** The paired plots also reveal strong model dependence. OPUS 4.5 benefits
1552 from 7 of the 9 compared variants, with an average ESR change of +2.72 percentage points. GPT-5.4
1553 benefits from 5 variants, averaging +2.22 percentage points. In contrast, GEMINI 2.5 PRO is harmed
1554 by 7 variants and averages −3.70 percentage points. Thus, the usefulness of procedural memory is
1555 not only a property of the memory mechanism; it also depends on the base model’s ability to interpret,
1556 select, and apply the provided memory.

1557 **Takeaway.** The cost–success analysis reinforces three main conclusions. First, skill abstraction is
1558 not automatically beneficial: static curated skills and sparse self-generated revision often add cost
1559 without improving ESR. Second, the most promising skill-based conditions are the always-update
1560 variants, but their gains remain model-specific and do not yet dominate direct trajectory retrieval.

1561 NeurIPS Paper Checklist

1562 1. Claims

1563 Question: Do the main claims made in the abstract and introduction accurately reflect the
1564 paper’s contributions and scope?

1565 Answer: [Yes]

1566 Justification: The abstract and introduction state that SKILLEVOLBENCH evaluates the step
1567 from episodic experience reuse to procedural skill formation, and the reported claims are
1568 supported by the benchmark design and experimental results in Sections 3–5. The claims are
1569 framed diagnostically and do not assert that current agents solve reusable skill formation.

1570 Guidelines:

- 1571 • The answer [N/A] means that the abstract and introduction do not include the claims
1572 made in the paper.
- 1573 • The abstract and/or introduction should clearly state the claims made, including the
1574 contributions made in the paper and important assumptions and limitations. A [No] or
1575 [N/A] answer to this question will not be perceived well by the reviewers.
- 1576 • The claims made should match theoretical and experimental results, and reflect how
1577 much the results can be expected to generalize to other settings.
- 1578 • It is fine to include aspirational goals as motivation as long as it is clear that these goals
1579 are not attained by the paper.

1580 2. Limitations

1581 Question: Does the paper discuss the limitations of the work performed by the authors?

1582 Answer: [Yes]

1583 Justification: The current draft discusses the limitations and future directions in Section 7.

1584 Guidelines:

- 1585 • The answer [N/A] means that the paper has no limitation while the answer [No] means
1586 that the paper has limitations, but those are not discussed in the paper.
- 1587 • The authors are encouraged to create a separate “Limitations” section in their paper.
- 1588 • The paper should point out any strong assumptions and how robust the results are to
1589 violations of these assumptions (e.g., independence assumptions, noiseless settings,
1590 model well-specification, asymptotic approximations only holding locally). The authors
1591 should reflect on how these assumptions might be violated in practice and what the
1592 implications would be.
- 1593 • The authors should reflect on the scope of the claims made, e.g., if the approach was
1594 only tested on a few datasets or with a few runs. In general, empirical results often
1595 depend on implicit assumptions, which should be articulated.
- 1596 • The authors should reflect on the factors that influence the performance of the approach.
1597 For example, a facial recognition algorithm may perform poorly when image resolution
1598 is low or images are taken in low lighting. Or a speech-to-text system might not be
1599 used reliably to provide closed captions for online lectures because it fails to handle
1600 technical jargon.
- 1601 • The authors should discuss the computational efficiency of the proposed algorithms
1602 and how they scale with dataset size.
- 1603 • If applicable, the authors should discuss possible limitations of their approach to
1604 address problems of privacy and fairness.
- 1605 • While the authors might fear that complete honesty about limitations might be used by
1606 reviewers as grounds for rejection, a worse outcome might be that reviewers discover
1607 limitations that aren’t acknowledged in the paper. The authors should use their best
1608 judgment and recognize that individual actions in favor of transparency play an impor-
1609 tant role in developing norms that preserve the integrity of the community. Reviewers
1610 will be specifically instructed to not penalize honesty concerning limitations.

1611 3. Theory assumptions and proofs

1612 Question: For each theoretical result, does the paper provide the full set of assumptions and
1613 a complete (and correct) proof?

Answer: [N/A]

Justification: The paper does not present theoretical results, theorems, or proofs. It is an empirical benchmark and diagnostic evaluation paper.

Guidelines:

- The answer [N/A] means that the paper does not include theoretical results.
- All the theorems, formulas, and proofs in the paper should be numbered and cross-referenced.
- All assumptions should be clearly stated or referenced in the statement of any theorems.
- The proofs can either appear in the main paper or the supplemental material, but if they appear in the supplemental material, the authors are encouraged to provide a short proof sketch to provide intuition.
- Inversely, any informal proof provided in the core of the paper should be complemented by formal proofs provided in appendix or supplemental material.
- Theorems and Lemmas that the proof relies upon should be properly referenced.

4. Experimental result reproducibility

Question: Does the paper fully disclose all the information needed to reproduce the main experimental results of the paper to the extent that it affects the main claims and/or conclusions of the paper (regardless of whether the code and data are provided or not)?

Answer: [Yes]

Justification: The paper specifies the benchmark construction, task roles, skill conditions, acquisition/deployment/replay protocol, scoring rule, evaluated harnesses and models, and experiment variants. Full reproducibility depends on the released task specifications, verifier suites, prompts, and execution harness configuration referenced in the appendix/supplement.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- If the paper includes experiments, a [No] answer to this question will not be perceived well by the reviewers: Making the paper reproducible is important, regardless of whether the code and data are provided or not.
- If the contribution is a dataset and/or model, the authors should describe the steps taken to make their results reproducible or verifiable.
- Depending on the contribution, reproducibility can be accomplished in various ways. For example, if the contribution is a novel architecture, describing the architecture fully might suffice, or if the contribution is a specific model and empirical evaluation, it may be necessary to either make it possible for others to replicate the model with the same dataset, or provide access to the model. In general, releasing code and data is often one good way to accomplish this, but reproducibility can also be provided via detailed instructions for how to replicate the results, access to a hosted model (e.g., in the case of a large language model), releasing of a model checkpoint, or other means that are appropriate to the research performed.
- While NeurIPS does not require releasing code, the conference does require all submissions to provide some reasonable avenue for reproducibility, which may depend on the nature of the contribution. For example
 - (a) If the contribution is primarily a new algorithm, the paper should make it clear how to reproduce that algorithm.
 - (b) If the contribution is primarily a new model architecture, the paper should describe the architecture clearly and fully.
 - (c) If the contribution is a new model (e.g., a large language model), then there should either be a way to access this model for reproducing the results or a way to reproduce the model (e.g., with an open-source dataset or instructions for how to construct the dataset).
 - (d) We recognize that reproducibility may be tricky in some cases, in which case authors are welcome to describe the particular way they provide for reproducibility. In the case of closed-source models, it may be that access to the model is limited in some way (e.g., to registered users), but it should be possible for other researchers to have some path to reproducing or verifying the results.

5. Open access to data and code

Question: Does the paper provide open access to the data and code, with sufficient instructions to faithfully reproduce the main experimental results, as described in supplemental material?

Answer: [Yes]

Justification: The code and dataset links are included in the submission information.

Guidelines:

- The answer [N/A] means that paper does not include experiments requiring code.
- Please see the NeurIPS code and data submission guidelines (<https://neurips.cc/public/guides/CodeSubmissionPolicy>) for more details.
- While we encourage the release of code and data, we understand that this might not be possible, so [No] is an acceptable answer. Papers cannot be rejected simply for not including code, unless this is central to the contribution (e.g., for a new open-source benchmark).
- The instructions should contain the exact command and environment needed to run to reproduce the results. See the NeurIPS code and data submission guidelines (<https://neurips.cc/public/guides/CodeSubmissionPolicy>) for more details.
- The authors should provide instructions on data access and preparation, including how to access the raw data, preprocessed data, intermediate data, and generated data, etc.
- The authors should provide scripts to reproduce all experimental results for the new proposed method and baselines. If only a subset of experiments are reproducible, they should state which ones are omitted from the script and why.
- At submission time, to preserve anonymity, the authors should release anonymized versions (if applicable).
- Providing as much information as possible in supplemental material (appended to the paper) is recommended, but including URLs to data and code is permitted.

6. Experimental setting/details

Question: Does the paper specify all the training and test details (e.g., data splits, hyperparameters, how they were chosen, type of optimizer) necessary to understand the results?

Answer: [Yes]

Justification: The paper describes the task construction process, role-conditioned progression, curated skill design, acquisition/deployment/replay protocol, scoring metrics, evaluated agent harnesses, model configurations, and experiment variants. Since this is an evaluation benchmark rather than a training method, there are no optimizer or training hyperparameters to report.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The experimental setting should be presented in the core of the paper to a level of detail that is necessary to appreciate the results and make sense of them.
- The full details can be provided either with the code, in appendix, or as supplemental material.

7. Experiment statistical significance

Question: Does the paper report error bars suitably and correctly defined or other appropriate information about the statistical significance of the experiments?

Answer: [No]

Justification: The current results are reported as success rates and percentage-point deltas without error bars, confidence intervals, or repeated-run variance estimates. This should be acknowledged because the experiments involve agentic model executions whose outcomes may vary across runs or model versions.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.

- The authors should answer [Yes] if the results are accompanied by error bars, confidence intervals, or statistical significance tests, at least for the experiments that support the main claims of the paper.
- The factors of variability that the error bars are capturing should be clearly stated (for example, train/test split, initialization, random drawing of some parameter, or overall run with given experimental conditions).
- The method for calculating the error bars should be explained (closed form formula, call to a library function, bootstrap, etc.)
- The assumptions made should be given (e.g., Normally distributed errors).
- It should be clear whether the error bar is the standard deviation or the standard error of the mean.
- It is OK to report 1-sigma error bars, but one should state it. The authors should preferably report a 2-sigma error bar than state that they have a 96% CI, if the hypothesis of Normality of errors is not verified.
- For asymmetric distributions, the authors should be careful not to show in tables or figures symmetric error bars that would yield results that are out of range (e.g., negative error rates).
- If error bars are reported in tables or plots, the authors should explain in the text how they were calculated and reference the corresponding figures or tables in the text.

8. Experiments compute resources

Question: For each experiment, does the paper provide sufficient information on the computer resources (type of compute workers, memory, time of execution) needed to reproduce the experiments?

Answer: [Yes]

Justification: The paper evaluates closed-source or hosted agent harnesses through API-based execution rather than local model training. The supplemental material reports the API providers, model configurations, execution setup, and estimated API costs needed to reproduce the benchmark runs.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The paper should indicate the type of compute workers CPU or GPU, internal cluster, or cloud provider, including relevant memory and storage.
- The paper should provide the amount of compute required for each of the individual experimental runs as well as estimate the total compute.
- The paper should disclose whether the full research project required more compute than the experiments reported in the paper (e.g., preliminary or failed experiments that didn't make it into the paper).

9. Code of ethics

Question: Does the research conducted in the paper conform, in every respect, with the NeurIPS Code of Ethics <https://neurips.cc/public/EthicsGuidelines>?

Answer: [Yes]

Justification: The author has reviewed the code of ethics.

Guidelines:

- The answer [N/A] means that the authors have not reviewed the NeurIPS Code of Ethics.
- If the authors answer [No], they should explain the special circumstances that require a deviation from the Code of Ethics.
- The authors should make sure to preserve anonymity (e.g., if there is a special consideration due to laws or regulations in their jurisdiction).

10. Broader impacts

Question: Does the paper discuss both potential positive societal impacts and negative societal impacts of the work performed?

Answer: [Yes]

Justification: The conclusion notes that the benchmark may support more reliable skill-evolving agents, while also identifying autonomy-related risks from agents that accumulate reusable procedures and the need for safeguards such as skill validation and update control.

Guidelines:

- The answer [N/A] means that there is no societal impact of the work performed.
- If the authors answer [N/A] or [No], they should explain why their work has no societal impact or why the paper does not address societal impact.
- Examples of negative societal impacts include potential malicious or unintended uses (e.g., disinformation, generating fake profiles, surveillance), fairness considerations (e.g., deployment of technologies that could make decisions that unfairly impact specific groups), privacy considerations, and security considerations.
- The conference expects that many papers will be foundational research and not tied to particular applications, let alone deployments. However, if there is a direct path to any negative applications, the authors should point it out. For example, it is legitimate to point out that an improvement in the quality of generative models could be used to generate Deepfakes for disinformation. On the other hand, it is not needed to point out that a generic algorithm for optimizing neural networks could enable people to train models that generate Deepfakes faster.
- The authors should consider possible harms that could arise when the technology is being used as intended and functioning correctly, harms that could arise when the technology is being used as intended but gives incorrect results, and harms following from (intentional or unintentional) misuse of the technology.
- If there are negative societal impacts, the authors could also discuss possible mitigation strategies (e.g., gated release of models, providing defenses in addition to attacks, mechanisms for monitoring misuse, mechanisms to monitor how a system learns from feedback over time, improving the efficiency and accessibility of ML).

11. Safeguards

Question: Does the paper describe safeguards that have been put in place for responsible release of data or models that have a high risk for misuse (e.g., pre-trained language models, image generators, or scraped datasets)?

Answer: [N/A]

Justification: The paper does not release pretrained models, image generators, scraped personal data, or other high-risk assets that require controlled release safeguards.

Guidelines:

- The answer [N/A] means that the paper poses no such risks.
- Released models that have a high risk for misuse or dual-use should be released with necessary safeguards to allow for controlled use of the model, for example by requiring that users adhere to usage guidelines or restrictions to access the model or implementing safety filters.
- Datasets that have been scraped from the Internet could pose safety risks. The authors should describe how they avoided releasing unsafe images.
- We recognize that providing effective safeguards is challenging, and many papers do not require this, but we encourage authors to take this into account and make a best faith effort.

12. Licenses for existing assets

Question: Are the creators or original owners of assets (e.g., code, data, models), used in the paper, properly credited and are the license and terms of use explicitly mentioned and properly respected?

Answer: [Yes]

Justification: Existing assets used or referenced in the work are credited through citations and URLs, including agent skill specifications, public skill repositories, related skill-oriented benchmarks, agent harnesses, and hosted model interfaces. The released benchmark and

code package will include license and attribution information for the new assets and will document any external dependencies, tools, or public resources required to reproduce the experiments.

Guidelines:

- The answer [N/A] means that the paper does not use existing assets.
- The authors should cite the original paper that produced the code package or dataset.
- The authors should state which version of the asset is used and, if possible, include a URL.
- The name of the license (e.g., CC-BY 4.0) should be included for each asset.
- For scraped data from a particular source (e.g., website), the copyright and terms of service of that source should be provided.
- If assets are released, the license, copyright information, and terms of use in the package should be provided. For popular datasets, paperswithcode.com/datasets has curated licenses for some datasets. Their licensing guide can help determine the license of a dataset.
- For existing datasets that are re-packaged, both the original license and the license of the derived asset (if it has changed) should be provided.
- If this information is not available online, the authors are encouraged to reach out to the asset's creators.

13. New assets

Question: Are new assets introduced in the paper well documented and is the documentation provided alongside the assets?

Answer: [Yes]

Justification: The paper introduces SKILLEVOLBENCH as a new benchmark and documents its environments, task families, role labels, task construction process, verification design, scoring metrics, and evaluation protocol.

Guidelines:

- The answer [N/A] means that the paper does not release new assets.
- Researchers should communicate the details of the dataset/code/model as part of their submissions via structured templates. This includes details about training, license, limitations, etc.
- The paper should discuss whether and how consent was obtained from people whose asset is used.
- At submission time, remember to anonymize your assets (if applicable). You can either create an anonymized URL or include an anonymized zip file.

14. Crowdsourcing and research with human subjects

Question: For crowdsourcing experiments and research with human subjects, does the paper include the full text of instructions given to participants and screenshots, if applicable, as well as details about compensation (if any)?

Answer: [N/A]

Justification: The work does not involve crowdsourcing experiments or human-subject studies.

Guidelines:

- The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
- Including this information in the supplemental material is fine, but if the main contribution of the paper involves human subjects, then as much detail as possible should be included in the main paper.
- According to the NeurIPS Code of Ethics, workers involved in data collection, curation, or other labor should be paid at least the minimum wage in the country of the data collector.

1876 **15. Institutional review board (IRB) approvals or equivalent for research with human**
1877 **subjects**

1878 Question: Does the paper describe potential risks incurred by study participants, whether
1879 such risks were disclosed to the subjects, and whether Institutional Review Board (IRB)
1880 approvals (or an equivalent approval/review based on the requirements of your country or
1881 institution) were obtained?

1882 Answer: [N/A]

1883 Justification: The work does not involve human-subject research, crowdsourced data col-
1884 lection, user studies, interviews, surveys, or experiments on participants. Therefore IRB
1885 approval or an equivalent human-subjects review is not applicable.

1886 Guidelines:

- 1887 • The answer [N/A] means that the paper does not involve crowdsourcing nor research
1888 with human subjects.
- 1889 • Depending on the country in which research is conducted, IRB approval (or equivalent)
1890 may be required for any human subjects research. If you obtained IRB approval, you
1891 should clearly state this in the paper.
- 1892 • We recognize that the procedures for this may vary significantly between institutions
1893 and locations, and we expect authors to adhere to the NeurIPS Code of Ethics and the
1894 guidelines for their institution.
- 1895 • For initial submissions, do not include any information that would break anonymity (if
1896 applicable), such as the institution conducting the review.

1897 **16. Declaration of LLM usage**

1898 Question: Does the paper describe the usage of LLMs if it is an important, original, or
1899 non-standard component of the core methods in this research? Note that if the LLM is used
1900 only for writing, editing, or formatting purposes and does *not* impact the core methodology,
1901 scientific rigor, or originality of the research, declaration is not required.

1902 Answer: [Yes]

1903 Justification: LLM agents and a host-side Skill Author call are central to the benchmark
1904 protocol. The paper describes how agent harnesses solve tasks, how compacted trajectories
1905 and verifier feedback are passed to the Skill Author, and how skill updates are written,
1906 revised, skipped, frozen, and evaluated.

1907 Guidelines:

- 1908 • The answer [N/A] means that the core method development in this research does not
1909 involve LLMs as any important, original, or non-standard components.
- 1910 • Please refer to our LLM policy in the NeurIPS handbook for what should or should not
1911 be described.